



香港科技大學  
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

# **ELEC 3310**

# **Digital Fundamentals and System Design**

**Wenchao Miao**

# Outline Lecture 5 Sequential Logic



THE HONG KONG  
UNIVERSITY OF SCIENCE  
AND TECHNOLOGY

**1 Learning Outcomes**

**2 Combinational vs Sequential Logic**

**3 Memory Elements: Latches and Flip-Flops**

**4 Timing Parameters**

**5 Traffic Signal Controller Design**

# 1. Learning Outcomes

- 1. Describe the difference between combinational and sequential logic**
- 2. Describe the need for the sequential logic**
- 3. Describe latches and flip-flops**
- 4. Describe level-triggered versus edge-triggered**
- 5. Describe synchronous and asynchronous inputs**
- 6. Define propagation delay time, set-up time, hold time of flip flops**
- 7. Design a traffic signal controller (sequential part)**

## Main Content

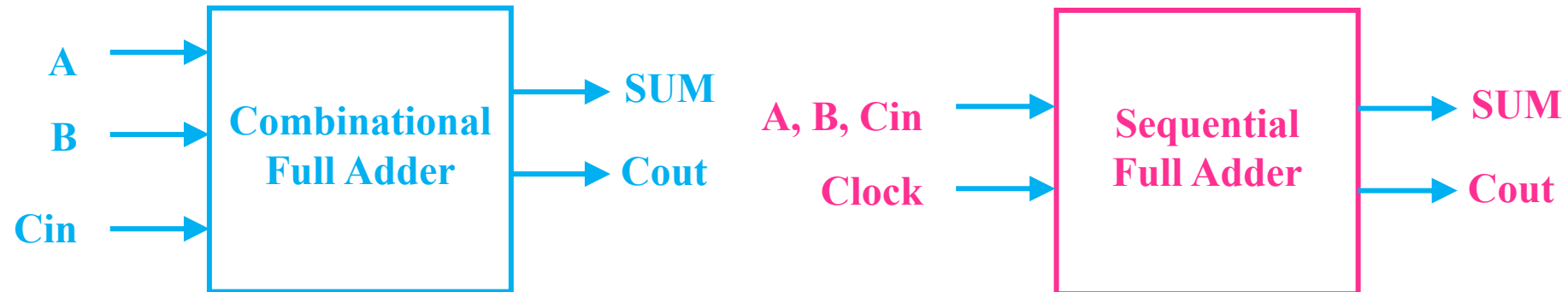
### Sequential Logic Fundamentals

- **Full Adder and Comparator: Combinational vs Sequential Implementations**
- **16-Bit Comparator and 4-Bit Pattern Detector: Sequential Logic Advantages**

## 2. Combinational vs Sequential Logic



### Example – I: Full Adder ( $F = A + B + C_{in}$ )



#### COMBINATIONAL Full Adder:

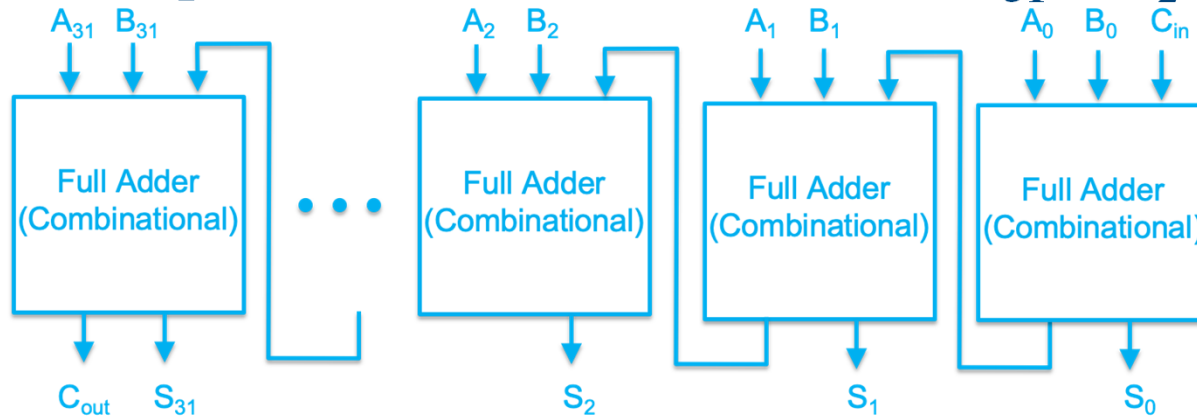
- **Three data inputs:** A, B, Cin supplied at the same time
- **One control input:** Enable (OPTIONAL)
- **Two outputs:** SUM & Cout fixed as long as inputs remain same change only when input changes
- **Contains/requires NO memory**
- **Outputs depend on the combination of (present) inputs**  
hence combinational logic

#### SEQUENTIAL (or serial) Full Adder:

- **One data input:** A, B, Cin supplied sequentially one at a time
- **One control input:** CLOCK (MUST)
- **Two outputs:** SUM & Cout fixed within the same clock cycle change with the clock cycle
- **Contains/requires memory**
- **Outputs depend on the sequence of past and present inputs**  
hence sequential logic

# Why Sequential Logic?

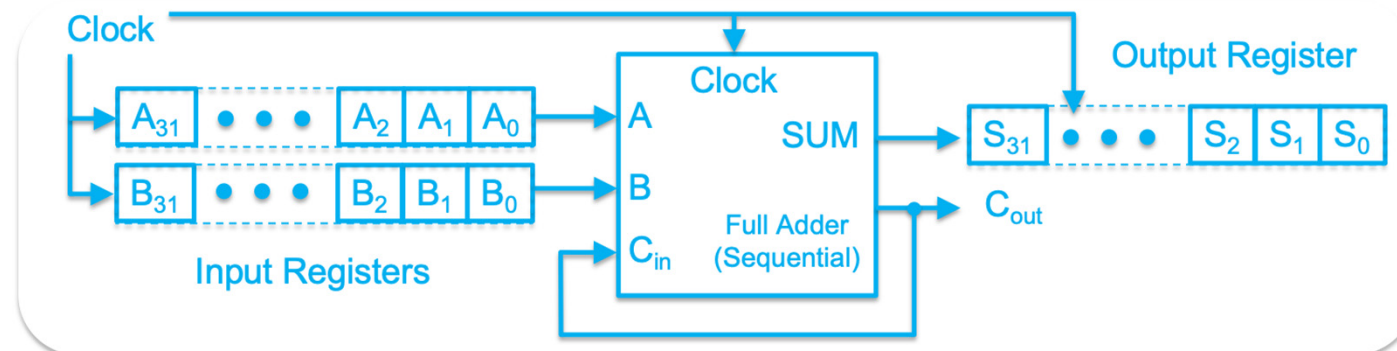
**Example – I: 32-bit full Adder ( $A=A_{31}...A_2A_1A_0$ ,  $B=B_{31}...B_2B_1B_0$ )**



**RESULT:**  
 $A + B = C_{out}S_{31}...S_2S_1S_0$

- Combinational Adder requires 32 full adders
- Sequential Adder requires only 1 full adder and additional registers (memory)
- Clock controls all (shift) registers and sequential adder block

**Note: this sequential adder slightly differs from previous example, and yet it still has limited number of inputs for the problem**



## 2. Combinational vs Sequential Logic

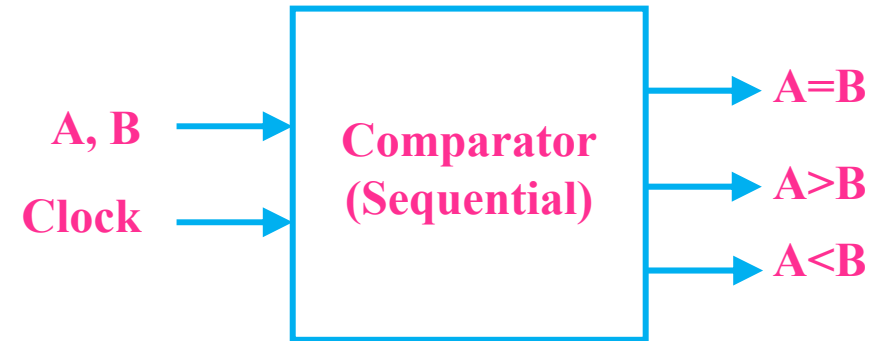


### Example – II: 2-bit Comparator ( $F \rightarrow A = B, A > B, A < B$ )



#### COMBINATIONAL Comparator:

- Two data inputs: A, B  
supplied at the same time
- One control input: Enable (OPTIONAL)
- Three outputs: =, >, and <  
fixed as long as inputs remain same  
change only when input changes
- Contains/requires NO memory
- Outputs depend on the combination of  
(present) inputs  
hence combinational logic

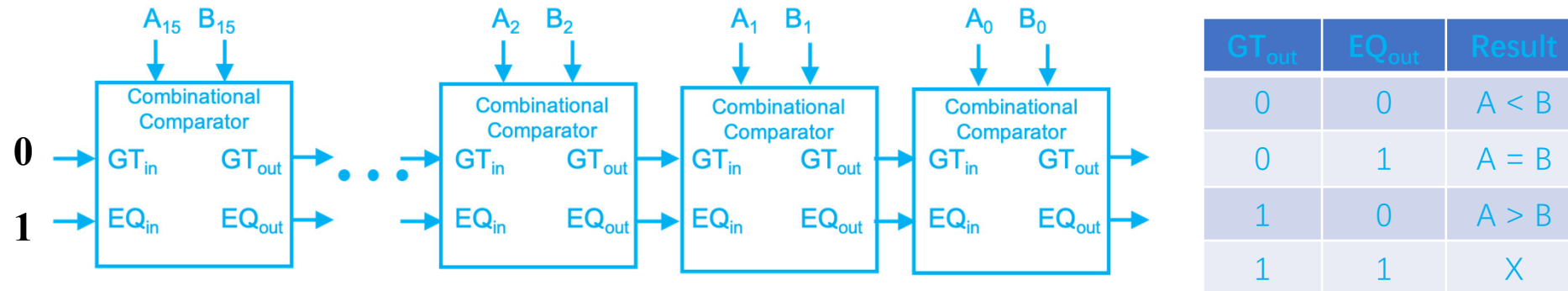


#### SEQUENTIAL (or serial) Comparator:

- One data input: A, B  
supplied sequentially one at a time
- One control input: CLOCK (MUST)
- Three outputs: =, >, and <  
fixed within the same clock cycle  
change with the clock cycle
- Contains/requires memory
- Outputs depend on the sequence of past and  
present inputs  
hence sequential logic

# Why Sequential Logic?

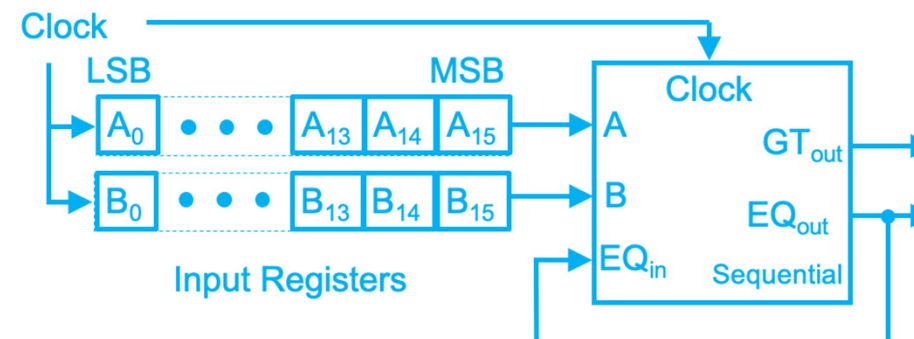
## Example – II: 16-bit comparator ( $A=A_{15}...A_2A_1A_0$ , $B=B_{15}...B_2B_1B_0$ )



**X → Don't care. This case won't be encountered**

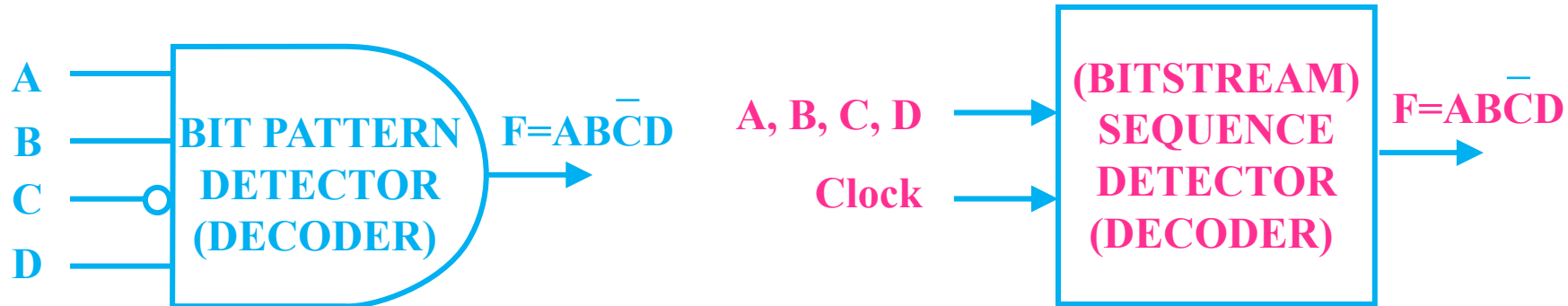
- Combinational logic requires 16 comparators
- Sequential logic requires only 1 comparator and some registers (memory) to retain bits
- Clock controls all (shift) registers and sequential comparator block
- If  $EQ_{in} = 0$  at any stage or clock cycle, then we can say either  $A > B$  or  $A < B$  for sure
  - No need to compare lower order bits

**Note: this sequential comparator slightly differs from previous example, and yet it still has limited number of inputs for the problem**



## 2. Combinational vs Sequential Logic

**Example – III: 4-bit pattern/bitstream detector (ABCD =? 1101)**



### BIT PATTERN DETECTOR (COMBINATIONAL)

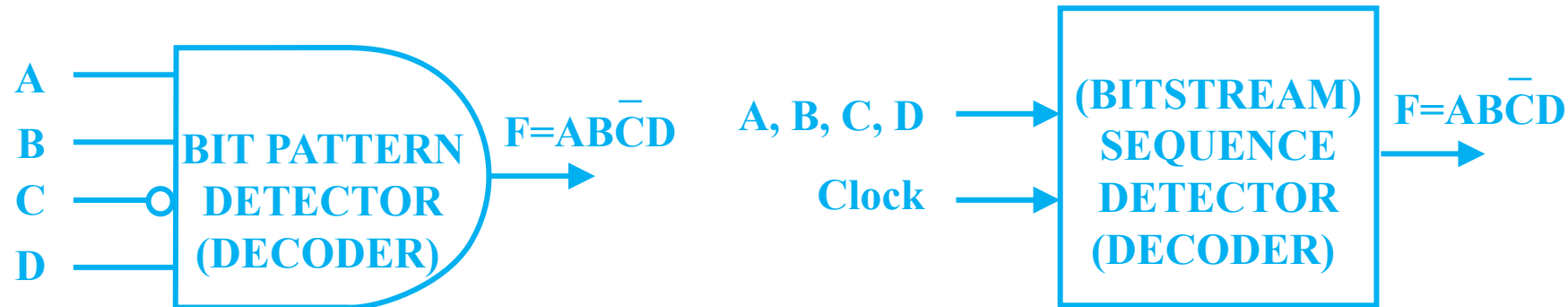
- Four data inputs: A, B, C, D supplied at the same time
- One control input: Enable (OPTIONAL)
- One output:  $F=1$  for  $ABCD=1101$  fixed as long as inputs remain same change only when input changes
- Contains/requires NO memory
- Outputs depend on the combination of (present) inputs hence combinational logic

### SEQUENCE DETECTOR (SEQUENTIAL)

- One data input: A, B, C, D supplied sequentially one at a time
- One control input: Clock (MUST)
- One output:  $F=1$  for  $ABCD=1101$  fixed within the same clock cycle change with the clock cycle
- Contains/requires memory
- Outputs depend on the sequence of past and present inputs hence sequential logic

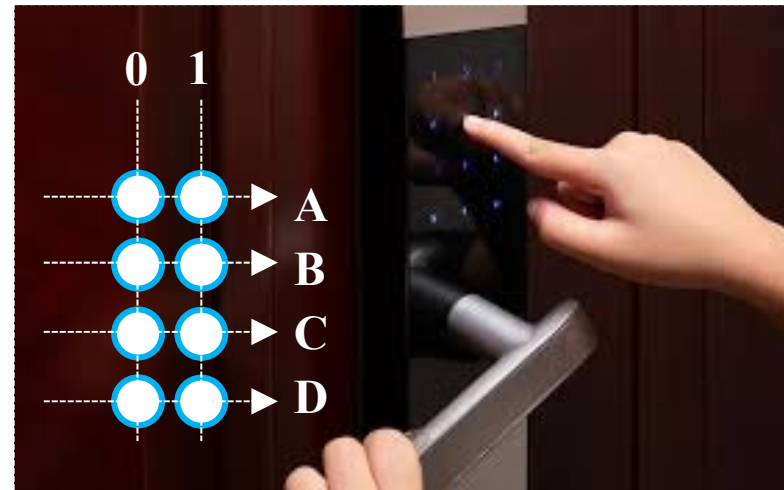
# Why Sequential Logic?

## Example – III: 4-bit pattern/bitstream detector (ABCD =? 1101)



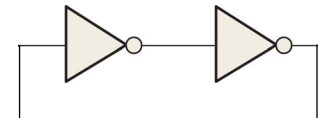
**Imagine a simple digital lock which requires a 4-bit code to unlock (1101)**

- **Combinational Logic:** must press all four buttons at the same time
- **Sequential Logic:** press four buttons in sequence



## 2. Combinational vs Sequential Logic

- Requires memory components (REGISTERS) to store bits
  - *This is because bits are input or used (processed) sequentially (one at a time)*
  - **Memory** function is achieved through **FEEDBACK** (connect the output of a logic circuit back to its input. Example: a loop made of inverters in cascade)
- Requires a **CLOCK** signal to control (to synchronize) all registers
  - **Clock** determines each sequence



## Main Content

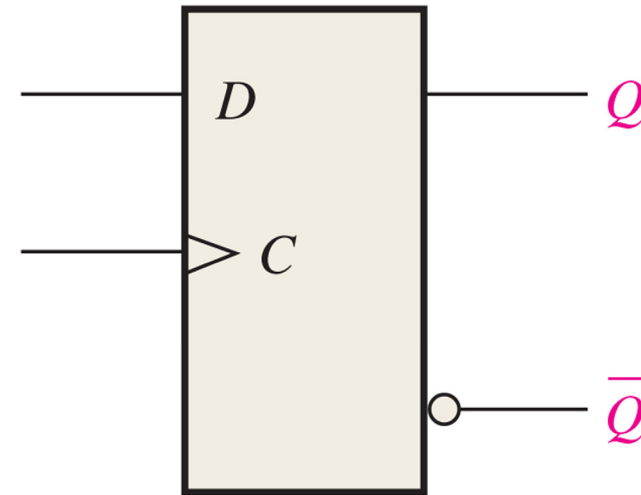
### Storage Components in Sequential

- Memory Requirements: Feedback and Clock Synchronization
- D Flip-Flop Operation (Edge-Triggered Behavior)
- D Latch vs D Flip-Flop: Level-Sensitive vs Edge-Sensitive
- IC Implementations (74HC75 Latch, 74HC74 Flip-Flop, Asynchronous Inputs)

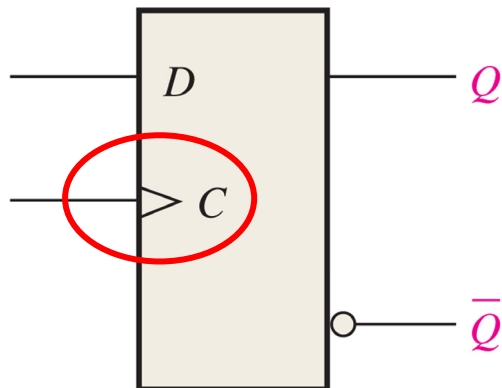
# 3. Memory Elements: Latches and Flip-Flops

## Memory: Storing single bits

- Memory component for single-bit storage is called “flip-flop” (there is also “latch”)
- We will mainly focus on “D flip-flop” (other types: JK, T flip-flops)
- Two inputs: one data input (“D” means data) and one control input (“C” means Clock)
- Two outputs (Q and  $\bar{Q}$ ) represent the bit stored (either 1 or 0)
- We will use D flip-flop for now and study internal logic later



- Note that the truth table shows Clock input
  - It will be implicit (will not be shown) in future truth tables of sequential logic
  - The Clock input is NOT explicitly shown in sequential logic
- Arrow “UP” means output Q changes with the rising edge of the clock
  - The rising clock edge is also known as “positive edge”
  - The falling clock edge is known as “negative edge”
- Triangle in the Clock input (logic symbol) indicates the flip-flop is “edge sensitive” (output transition occurs with the clock edge)
- If no triangle, then “level sensitive” meaning a latch (NOT a flip-flop)
  - It is called “D Latch” and Clock input works like Enable input



Truth table for a positive edge-triggered D flip-flop.

| Inputs |     | Outputs |           | Comments |
|--------|-----|---------|-----------|----------|
| D      | CLK | Q       | $\bar{Q}$ |          |
| 0      | ↑   | 0       | 1         | RESET    |
| 1      | ↑   | 1       | 0         | SET      |

↑ = clock transition LOW to HIGH

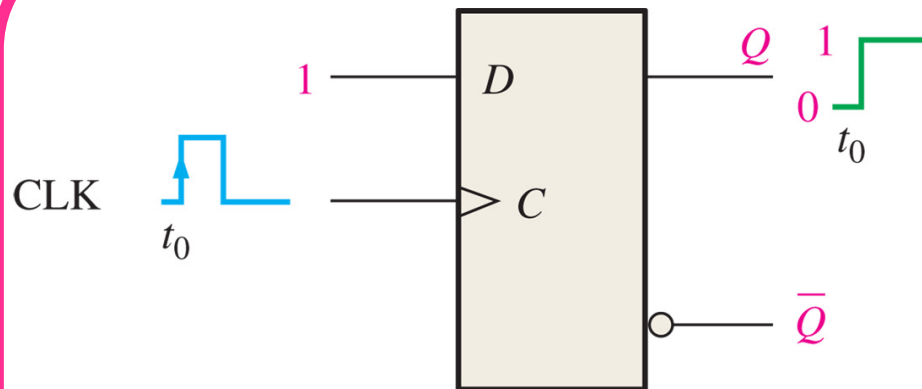
# D flip-flop operation

- **SET** means setting  $Q$  to binary 1
  - **D input has to be 1 just before clock edge**
- **RESET** means setting  $Q$  to binary 0
  - **D input has to be 0 just before clock edge**
- **SET and RESET are used in sequential logic quite often!**

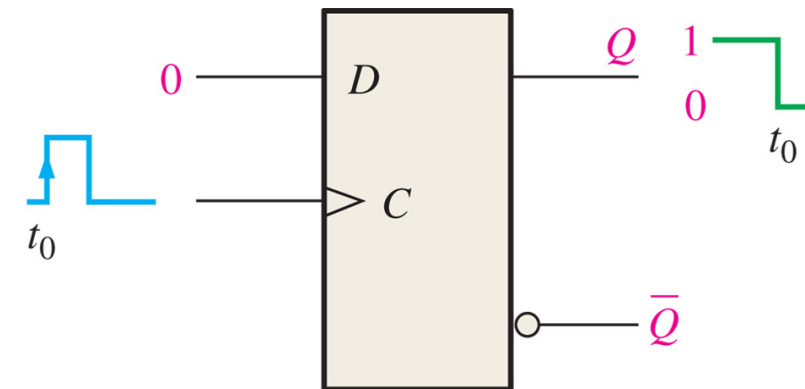
Truth table for a positive edge-triggered D flip-flop.

| Inputs |     | Outputs |           | Comments |
|--------|-----|---------|-----------|----------|
| $D$    | CLK | $Q$     | $\bar{Q}$ |          |
| 0      | ↑   | 0       | 1         | RESET    |
| 1      | ↑   | 1       | 0         | SET      |

↑ = clock transition LOW to HIGH

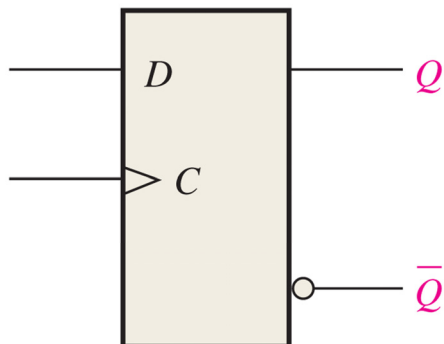
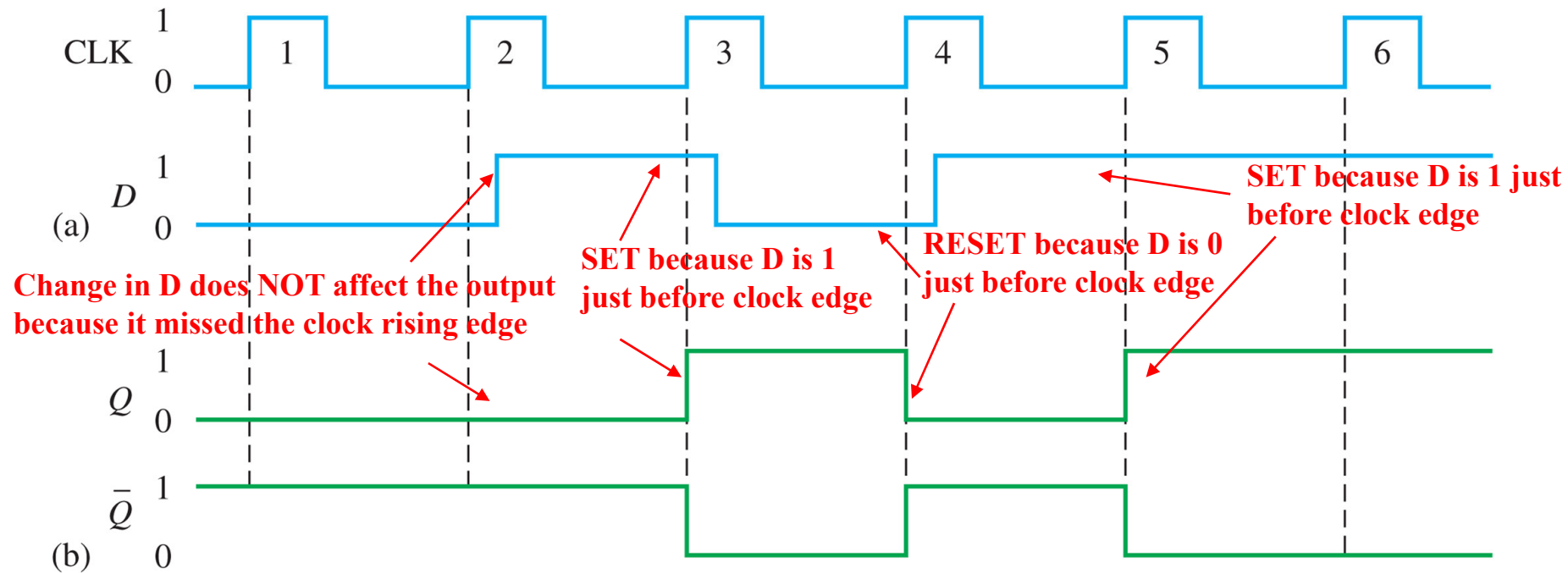


(a)  $D = 1$  flip-flop SETS on positive clock edge. (If already SET, it remains SET.)



(b)  $D = 0$  flip-flop RESETS on positive clock edge. (If already RESET, it remains RESET.)

# D flip-flop: Timing diagram



Truth table for a positive edge-triggered D flip-flop.

| Inputs |     | Outputs |           | Comments |
|--------|-----|---------|-----------|----------|
| D      | CLK | Q       | $\bar{Q}$ |          |
| 0      | ↑   | 0       | 1         | RESET    |
| 1      | ↑   | 1       | 0         | SET      |

↑ = clock transition LOW to HIGH

# D flip-flop: storing a bit

- **SET operation stores binary 1**
- **RESET operation stores binary 0**

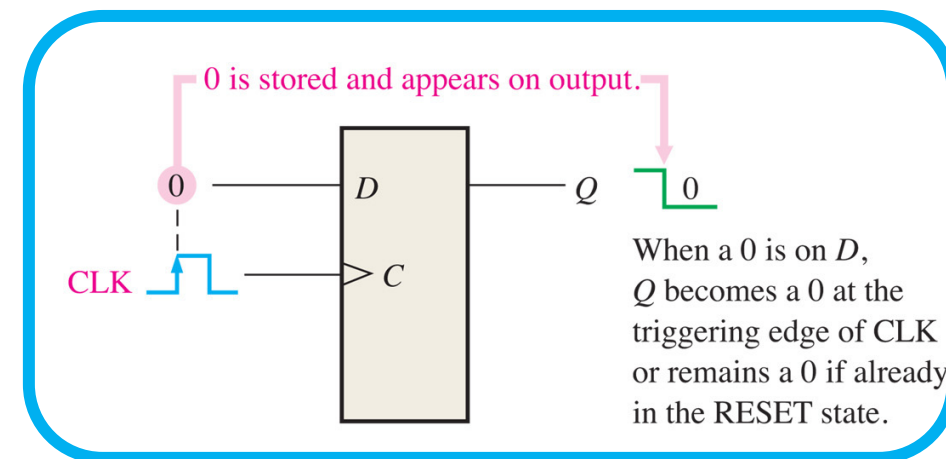
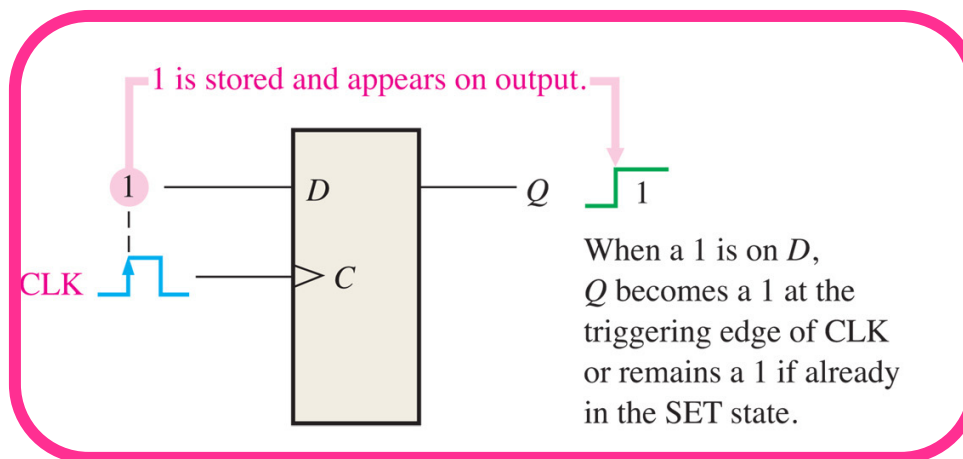
Truth table for a positive edge-triggered D flip-flop.

| Inputs |     | Outputs |           | Comments |
|--------|-----|---------|-----------|----------|
| $D$    | CLK | $Q$     | $\bar{Q}$ |          |
| 0      | ↑   | 0       | 1         | RESET    |
| 1      | ↑   | 1       | 0         | SET      |

↑ = clock transition LOW to HIGH

**SET and RESET operations are also known as the STATE of the flip-flop**

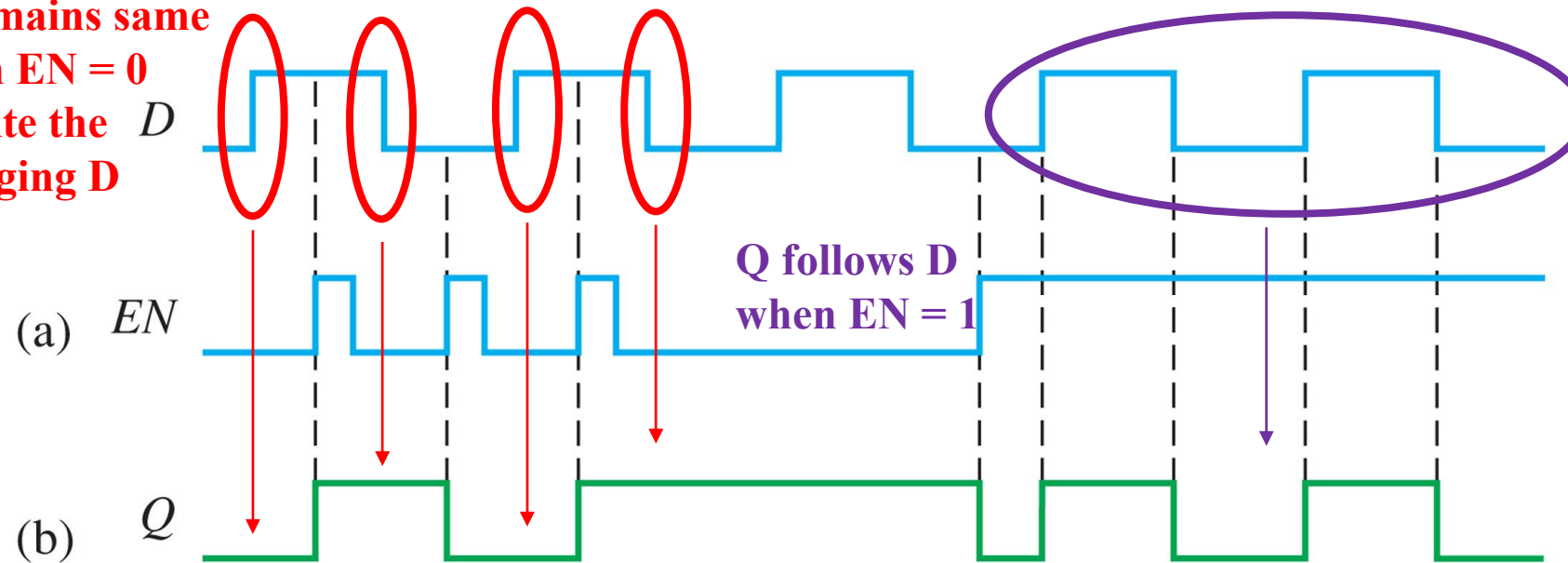
**The flip-flop is in RESET state ( $Q = 0$ ) or in SET state ( $Q = 1$ )**



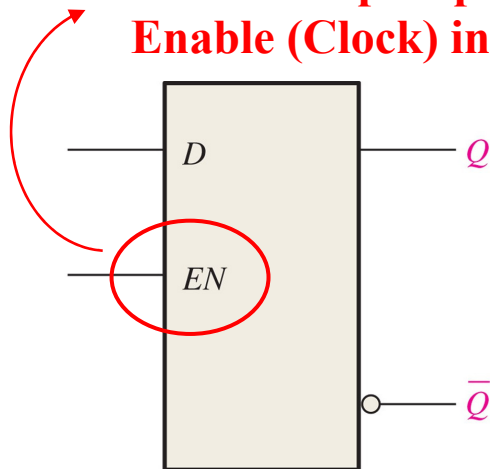
# D Latch: Timing diagram



**Q remains same  
when  $EN = 0$   
despite the  
changing D**



**Latch & flip-flop symbols are very similar except NO triangle for a latch at the Enable (Clock) input indicating that it's level-sensitive (NOT edge-sensitive)**



(b) Logic symbol

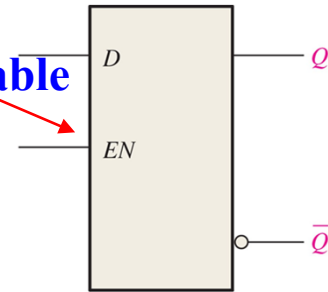
| Inputs |      | Outputs |           | Comments |
|--------|------|---------|-----------|----------|
| $D$    | $EN$ | $Q$     | $\bar{Q}$ |          |
| 0      | 1    | 0       | 1         | RESET    |
| 1      | 1    | 1       | 0         | SET      |

# D Latch and D Flip-flop



D-latch (level  
triggered)

NO triangle at Enable  
(clock) input

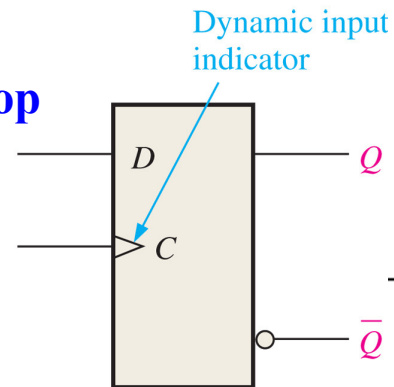


Enable

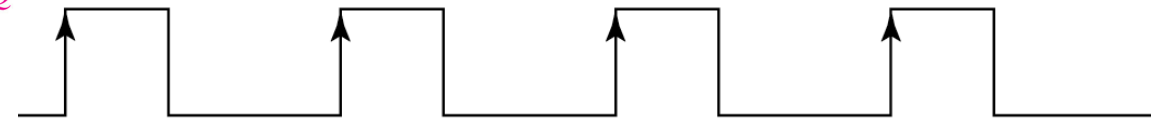


(a) Response to positive level

A positive-edge  
triggered D-flipflop

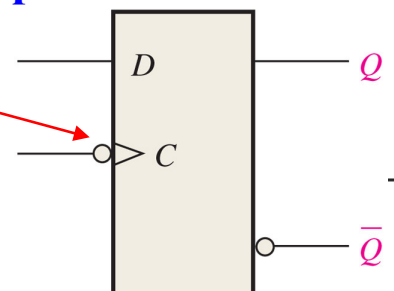


Clock



(b) Positive-edge response

A negative-edge  
triggered D-flipflop  
(bubble)

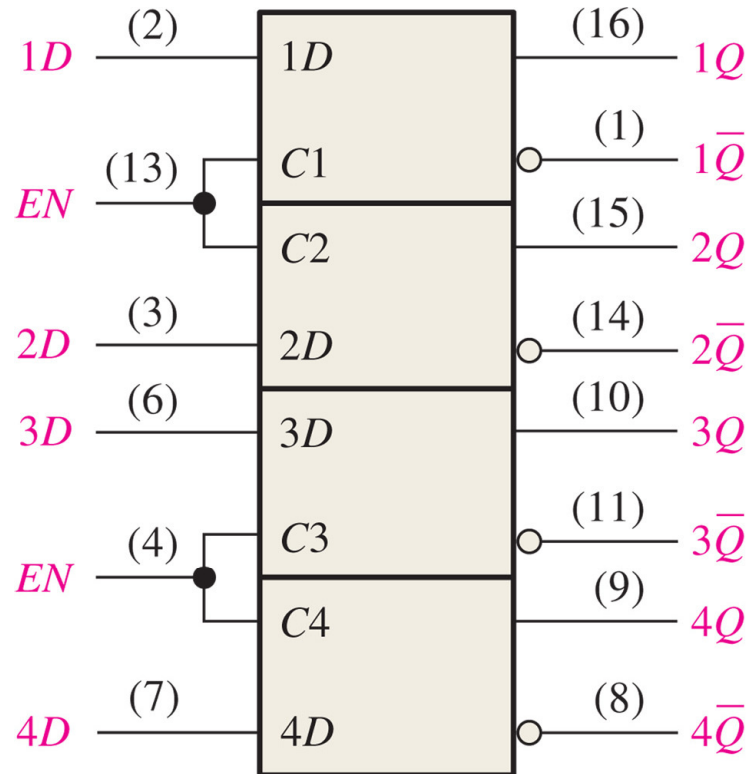


Clock



(c) Negative-edge response

# The 74HC75 quad D latch



(a) Logic symbol

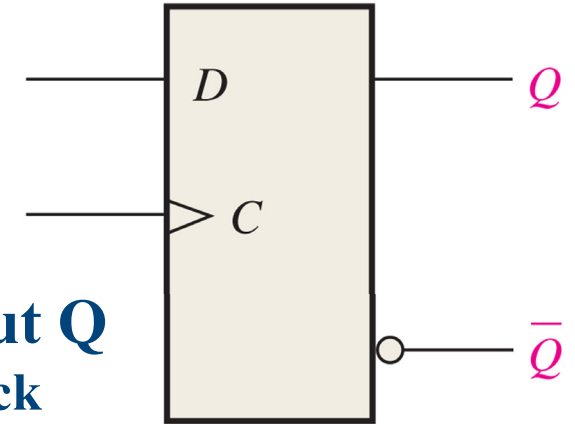
| Inputs |      | Outputs |             | Comments  |
|--------|------|---------|-------------|-----------|
| $D$    | $EN$ | $Q$     | $\bar{Q}$   |           |
| 0      | 1    | 0       | 1           | RESET     |
| 1      | 1    | 1       | 0           | SET       |
| X      | 0    | $Q_0$   | $\bar{Q}_0$ | No change |

*Note:*  $Q_0$  is the prior output level before the indicated input conditions were established.

(b) Truth table (each latch)

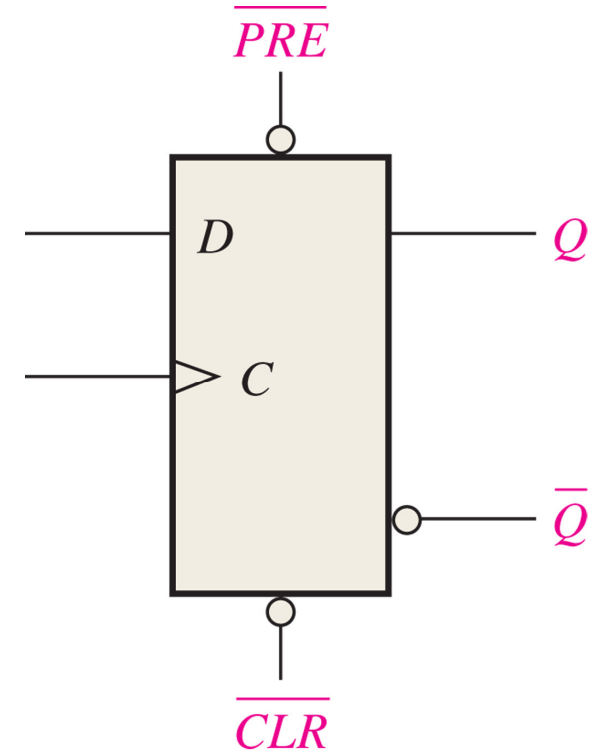
# Inputs: Synchronous vs Asynchronous

- **D input is a Synchronous input**
- **Synchronous and Asynchronous inputs**
  - defined in reference to Clock signal
- **Synchronous** inputs needs clock edge to change the output  $Q$ 
  - Change in the input does NOT change the output before the clock edge arrives
  - D input is “synchronous with the clock edge”
- **Asynchronous** inputs do NOT need clock edge
  - They can change the output  $Q$  independent of the clock edge
  - Thus, they have priority and can override synchronous inputs



# Asynchronous Inputs: Preset & Clear

- Preset and Clear inputs are **asynchronous** inputs
- Both are “**active LOW**” (bubbles) meaning:
  - Applying 0 to **Preset** input sets **Q to 1**
  - Applying 0 to **Clear** input sets **Q to 0**
- Both inputs **override D input (they have priority)**
- Keep both Preset and Clear inputs **HIGH (1)** during normal operation
- They give control to interfere with normal operation **when necessary**

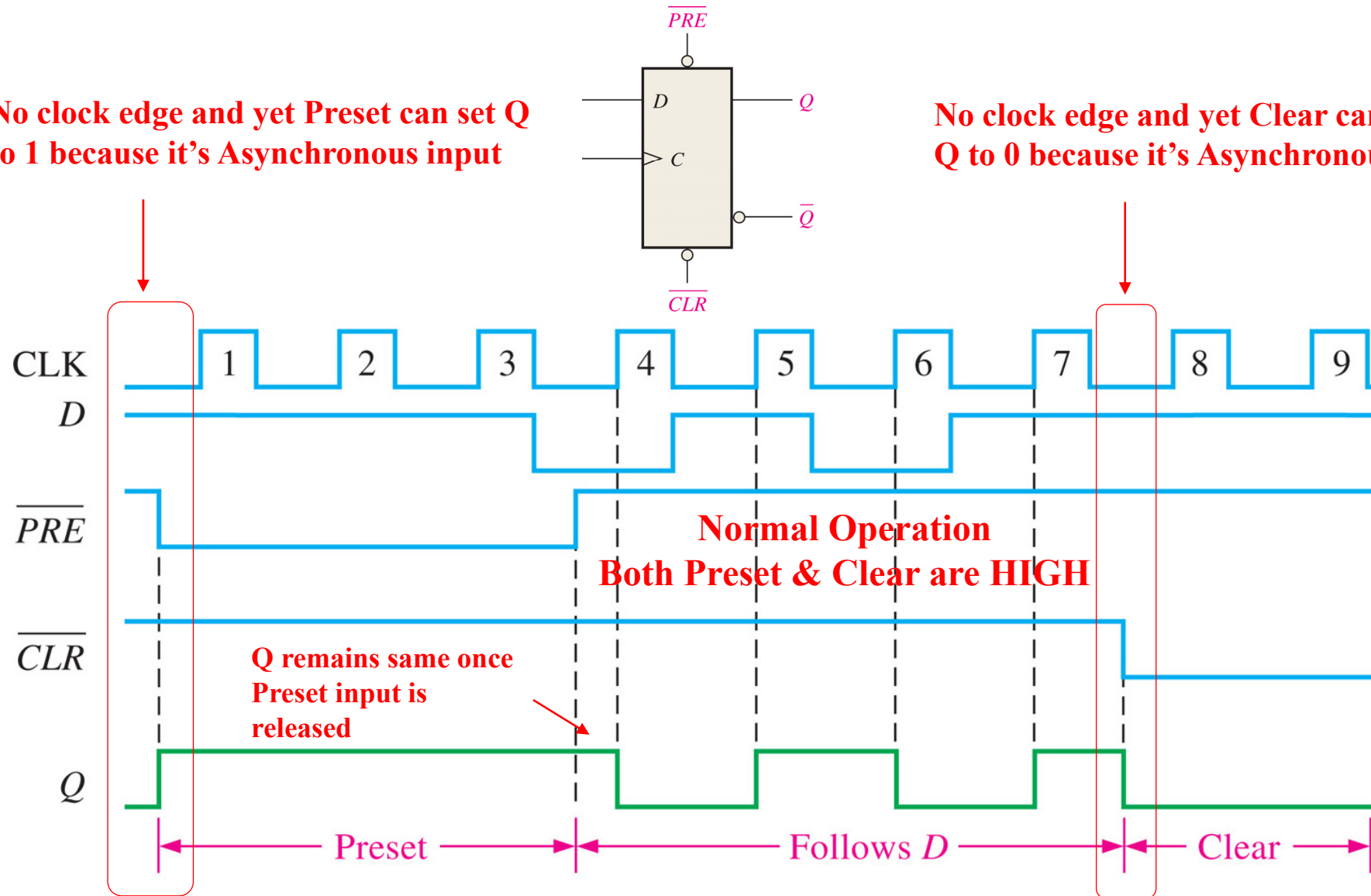


# D flip-flop: Timing diagram

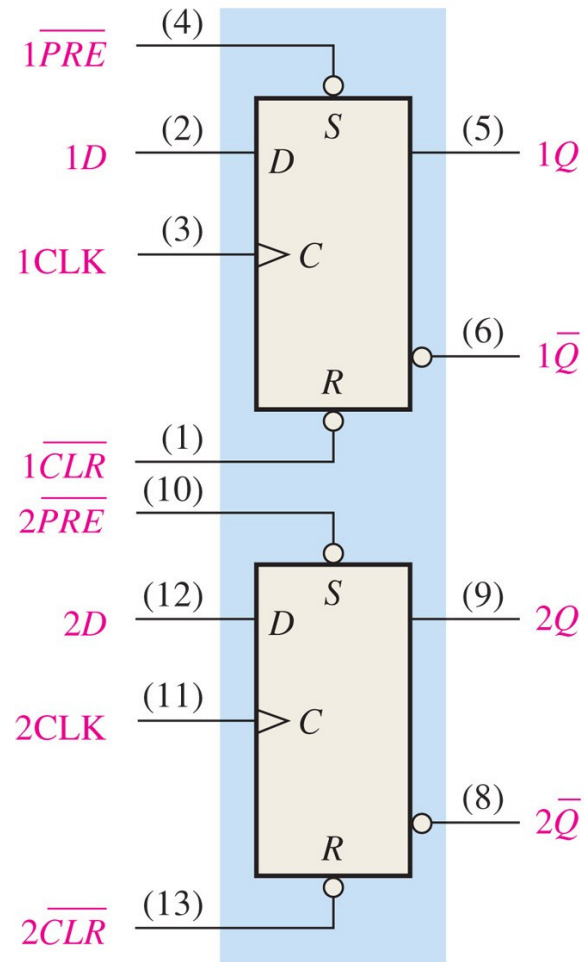


No clock edge and yet Preset can set  $Q$  to 1 because it's Asynchronous input

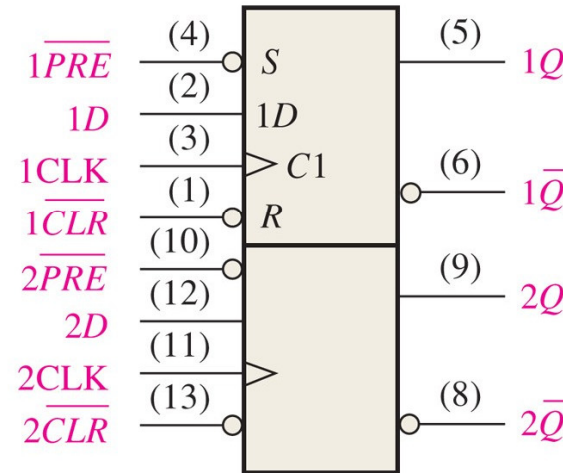
No clock edge and yet Clear can reset  $Q$  to 0 because it's Asynchronous input



# The 74HC74 dual positive edge-triggered D flip-flop



(a) Individual logic symbols



(b) Single block logic symbol

Note: The  $S$  and  $R$  inside the block indicate that  $\overline{PRE}$  SETS and  $\overline{CLR}$  RESETS.

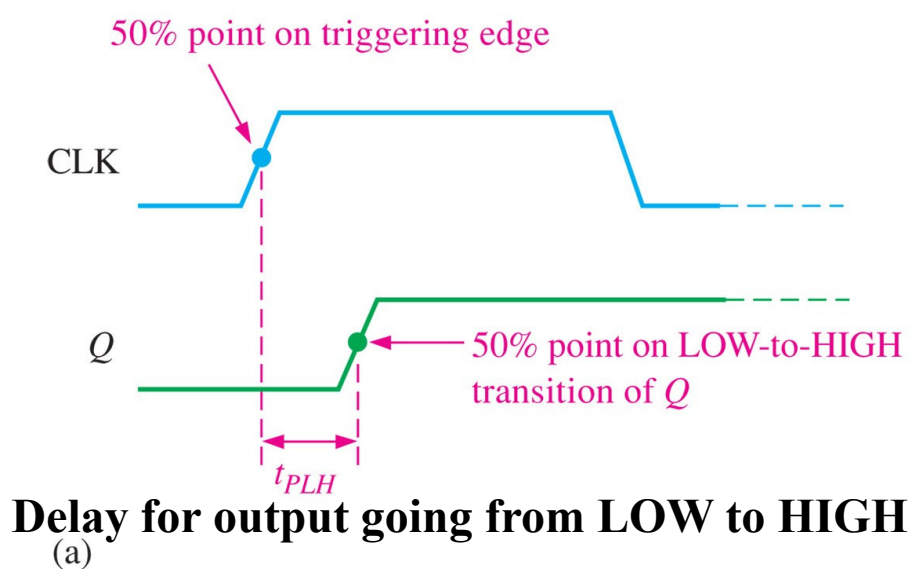
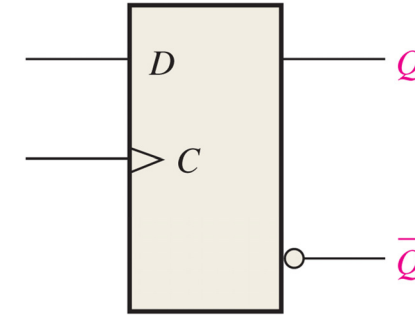
## Main Content

### Critical Timing Constraints

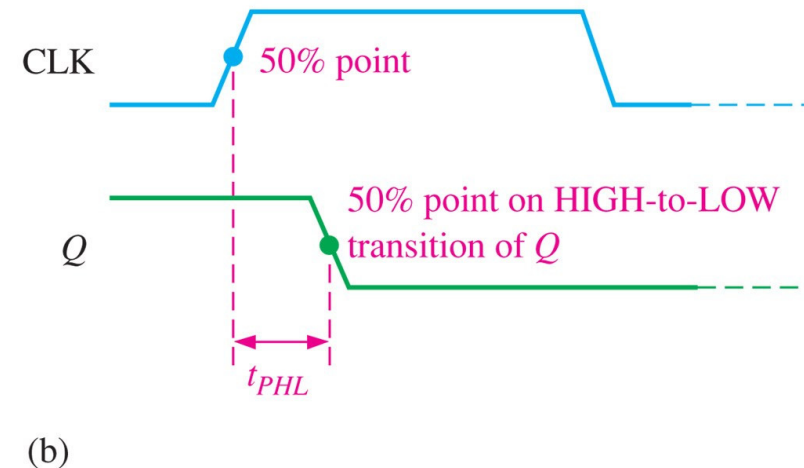
- Propagation Delays (CLK-to-Q, Preset/Clear-to-Q)
- Set-Up Time ( $t_s$ ) and Hold Time ( $t_h$ )
- IC Timing Comparison (CMOS vs TTL Families)

## Clock-to-output delay

- No such thing as “D input to output delay”
- For “synchronous” inputs we refer to “clock-to-output delay”

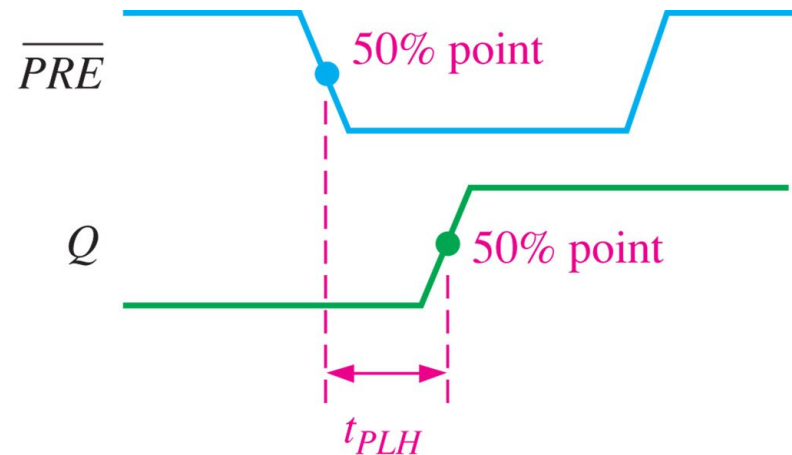
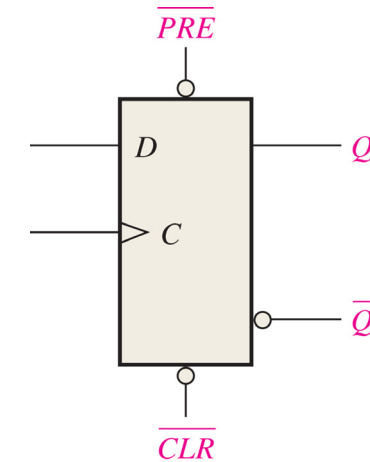


## Delay for output going from HIGH to LOW



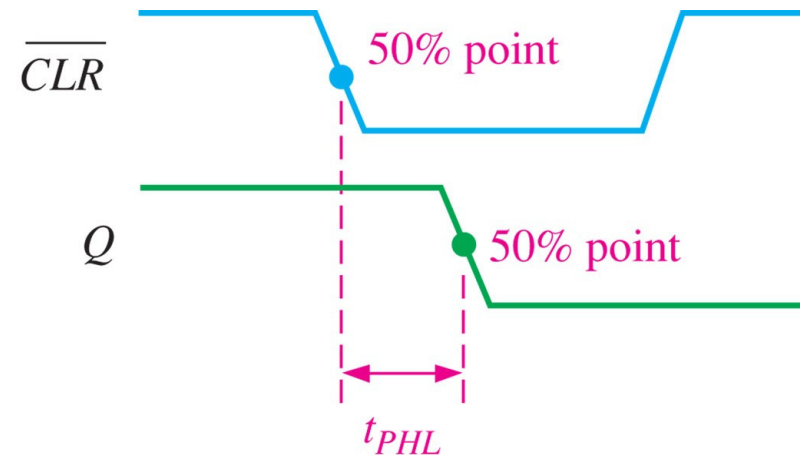
preset input to output delay  
clear input to output delay

- Both Preset and Clear inputs are Asynchronous inputs
- Asynchronous inputs are independent of Clock signal



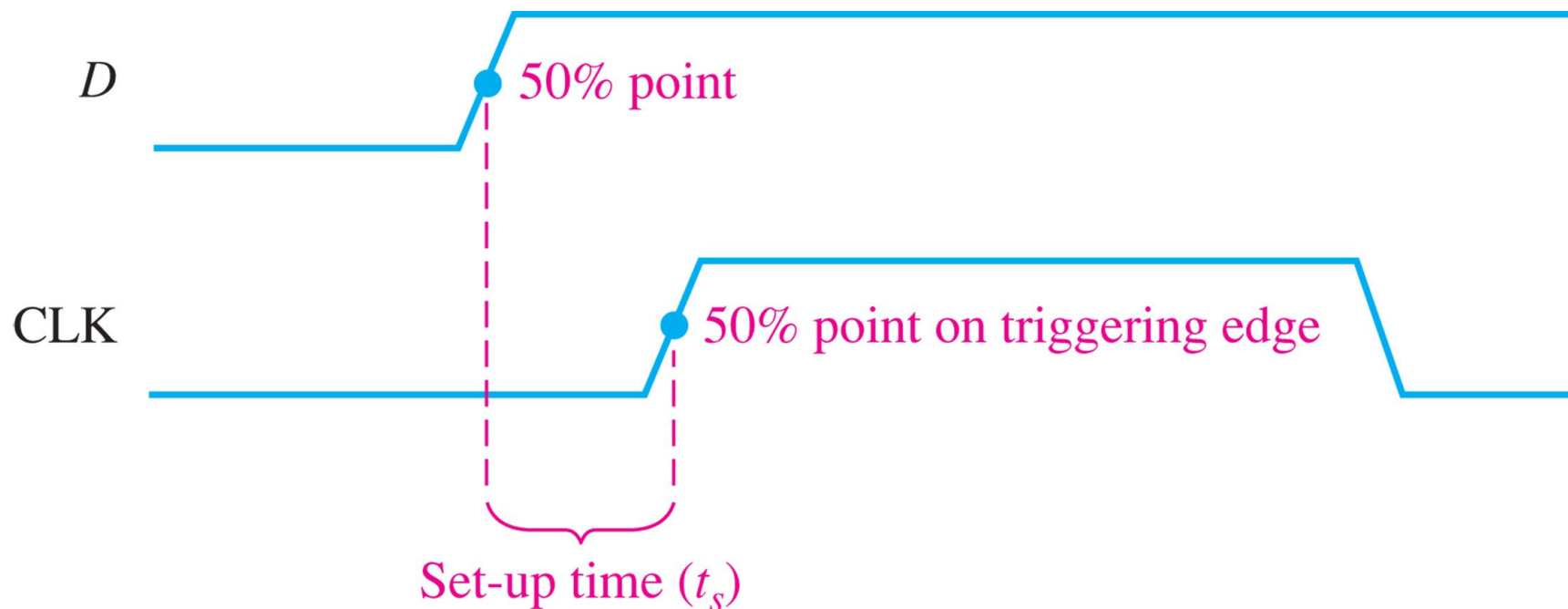
(a) Delay for output going from LOW to HIGH

Delay for output going from HIGH to LOW

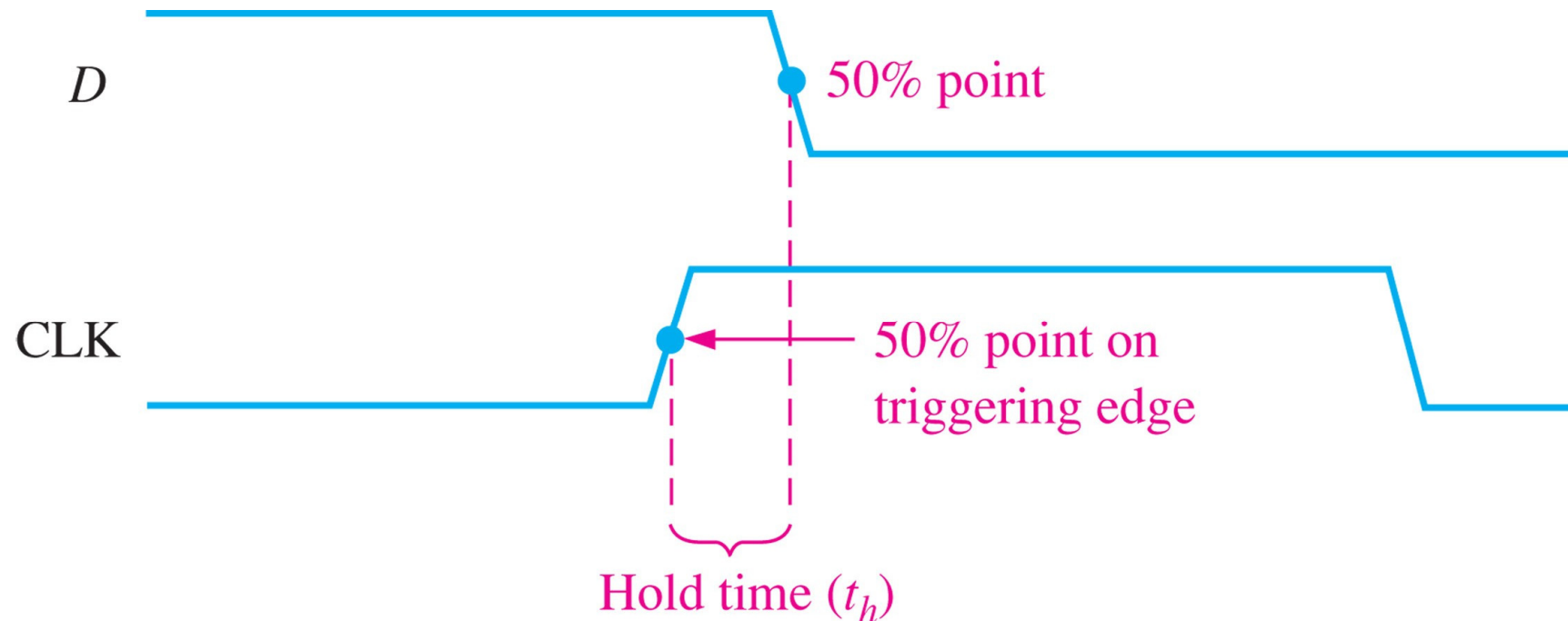


(b)

The logic level must be present on the  $D$  input for a time **equal to or greater than set-up time  $t_s$**  before the triggering edge of the clock pulse **for reliable data entry**



The logic level must remain on the  $D$  input for a time **equal to or greater than hold time  $t_h$**  after the triggering edge of the clock pulse **for reliable data entry**



**TABLE 7-4**

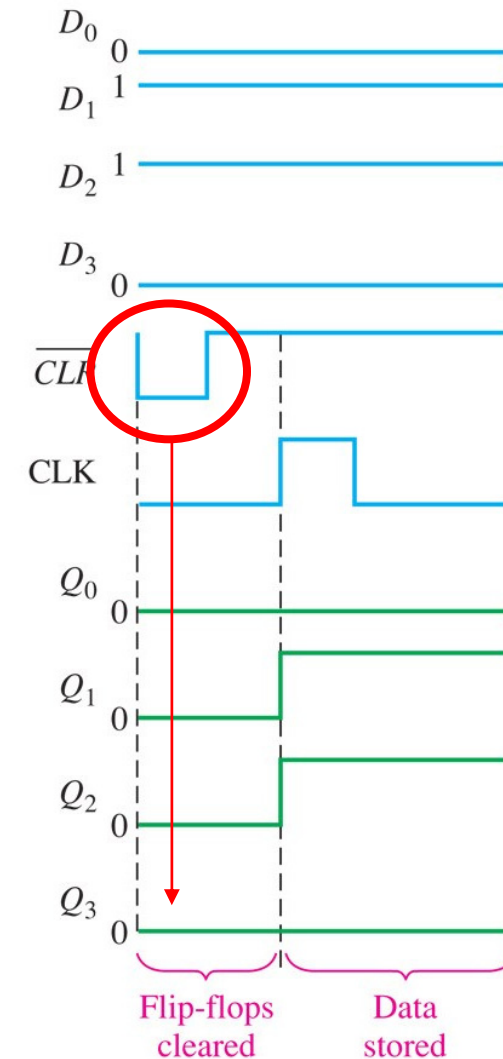
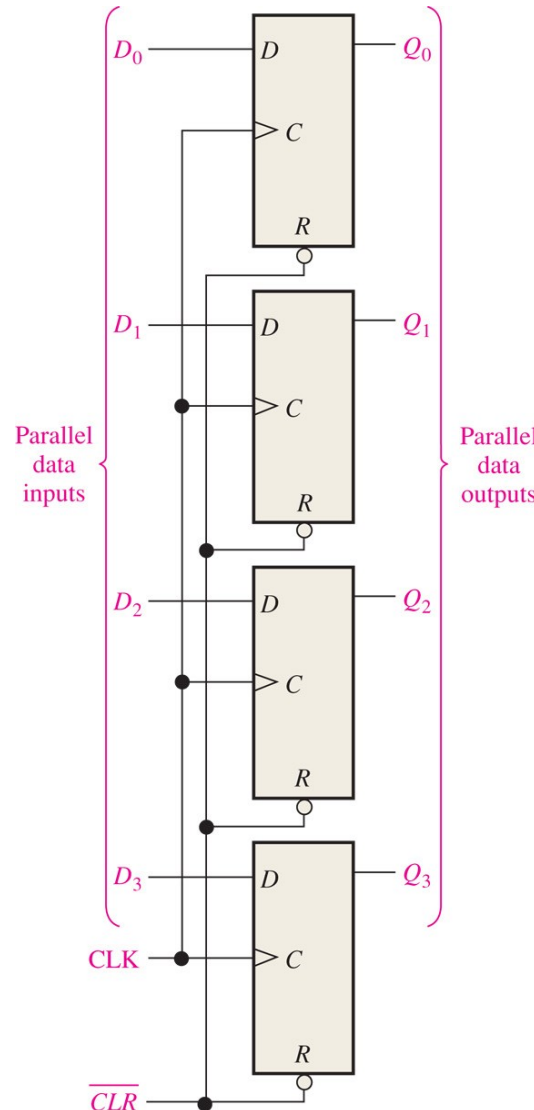
Comparison of operating parameters for four IC families of flip-flops of the same type at 25°C.

| Parameter                            | CMOS     |         | Bipolar (TTL) |         |
|--------------------------------------|----------|---------|---------------|---------|
|                                      | 74HC74A  | 74AHC74 | 74LS74A       | 74F74   |
| $t_{PHL}$ (CLK to $Q$ )              | 17 ns    | 4.6 ns  | 40 ns         | 6.8 ns  |
| $t_{PLH}$ (CLK to $Q$ )              | 17 ns    | 4.6 ns  | 25 ns         | 8.0 ns  |
| $t_{PHL}(\overline{CLR}$ to $Q$ )    | 18 ns    | 4.8 ns  | 40 ns         | 9.0 ns  |
| $t_{PLH}(\overline{PRE}$ to $Q$ )    | 18 ns    | 4.8 ns  | 25 ns         | 6.1 ns  |
| $t_s$ (set-up time)                  | 14 ns    | 5.0 ns  | 20 ns         | 2.0 ns  |
| $t_h$ (hold time)                    | 3.0 ns   | 0.5 ns  | 5 ns          | 1.0 ns  |
| $t_W$ (CLK HIGH)                     | 10 ns    | 5.0 ns  | 25 ns         | 4.0 ns  |
| $t_W$ (CLK LOW)                      | 10 ns    | 5.0 ns  | 25 ns         | 5.0 ns  |
| $t_W(\overline{CLR}/\overline{PRE})$ | 10 ns    | 5.0 ns  | 25 ns         | 4.0 ns  |
| $f_{max}$                            | 35 MHz   | 170 MHz | 25 MHz        | 100 MHz |
| Power, quiescent                     | 0.012 mW | 1.1 mW  |               |         |
| Power, 50% duty cycle                |          |         | 44 mW         | 88 mW   |

# Registers: Storing multiple bits

- Use as many flip-flops as the number of bits to store
- Parallel 4-bit register → parallel because writing (loading) and reading (recalling) bits are done in parallel
- PRESET inputs are implicit (not shown but we know they are connected to HIGH for normal operation)

*A single D flip-flop is a 1-bit register*



## Main Content

### Practical Application of Sequential Logic

- System Requirements and State Diagram (Gray Code Transitions)
- Sequential Logic Block Design: 2-Bit Gray Code Counter
- State Transitions, Next-State Tables, and Input Logic

Develop a control logic for a traffic signal at the intersection of a busy main street and a lightly used side street



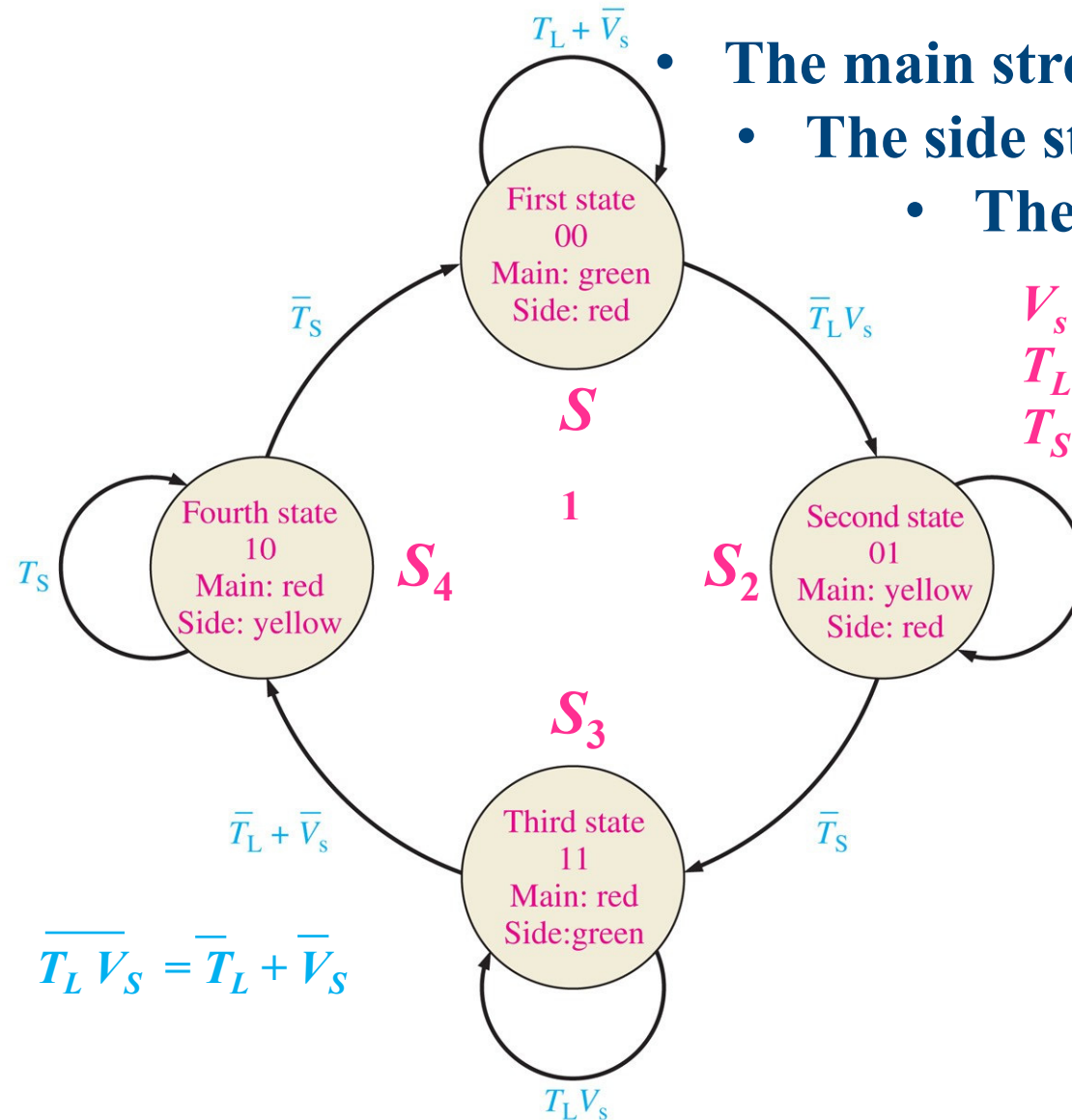
- The the main street green light will stay on for a minimum of 25 s OR as long as there is no vehicle on the side street.  $(T_L + \bar{V}_S)$
- The side street green light will stay on until there is no vehicle on the side street up to a maximum of 25 s.  $(T_L V_S)$
- The yellow caution light will stay on for 4 s between changes from green to red on both the main street and the side street.  $T_S$

$V_S$  A vehicle exists on the side street

$T_L$  The 25s timer (long timer) is ON

$T_S$  The 4s timer (short timer) is ON

# State diagram



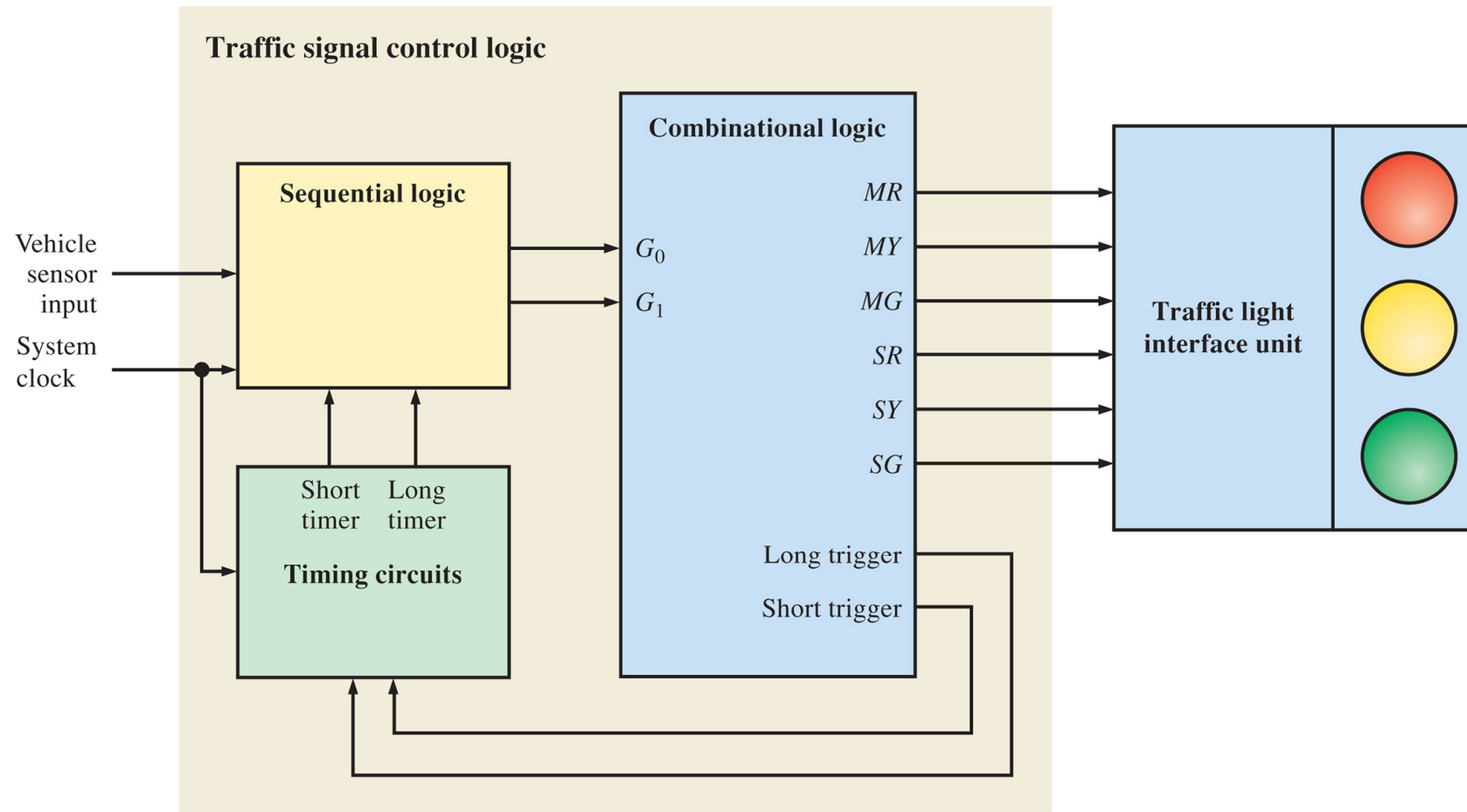
- The main street green light ON ( $T_L + \bar{V}_S$ )
- The side street green light ON ( $T_L V_S$ )
- The yellow caution light ON  $T_S$

$V_S$  A vehicle exists on the side street  
 $T_L$  The 25s timer (long timer) is ON  
 $T_S$  The 4s timer (short timer) is ON

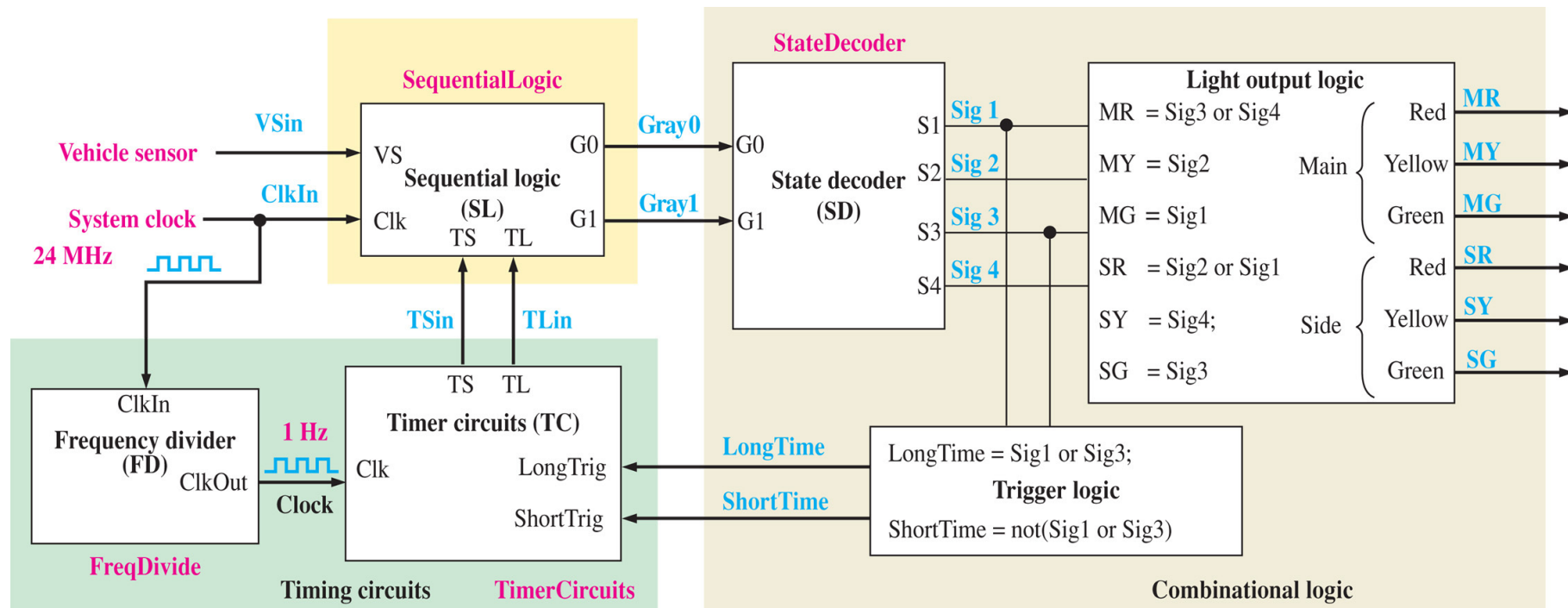
Note: each state is identified with a unique binary code (gray code)

$S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$   
 $00 \rightarrow 01 \rightarrow 11 \rightarrow 10$

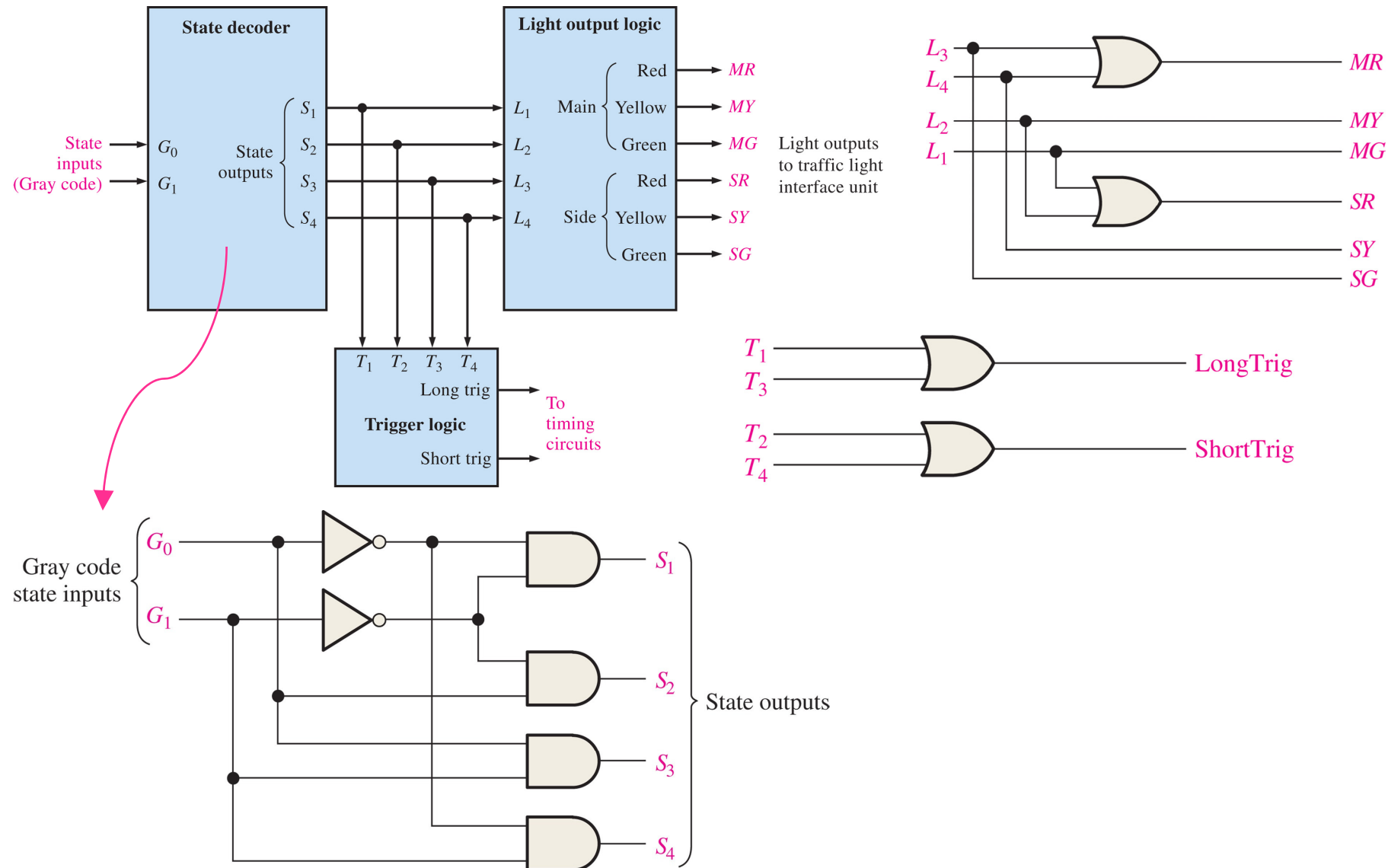
# Block diagram



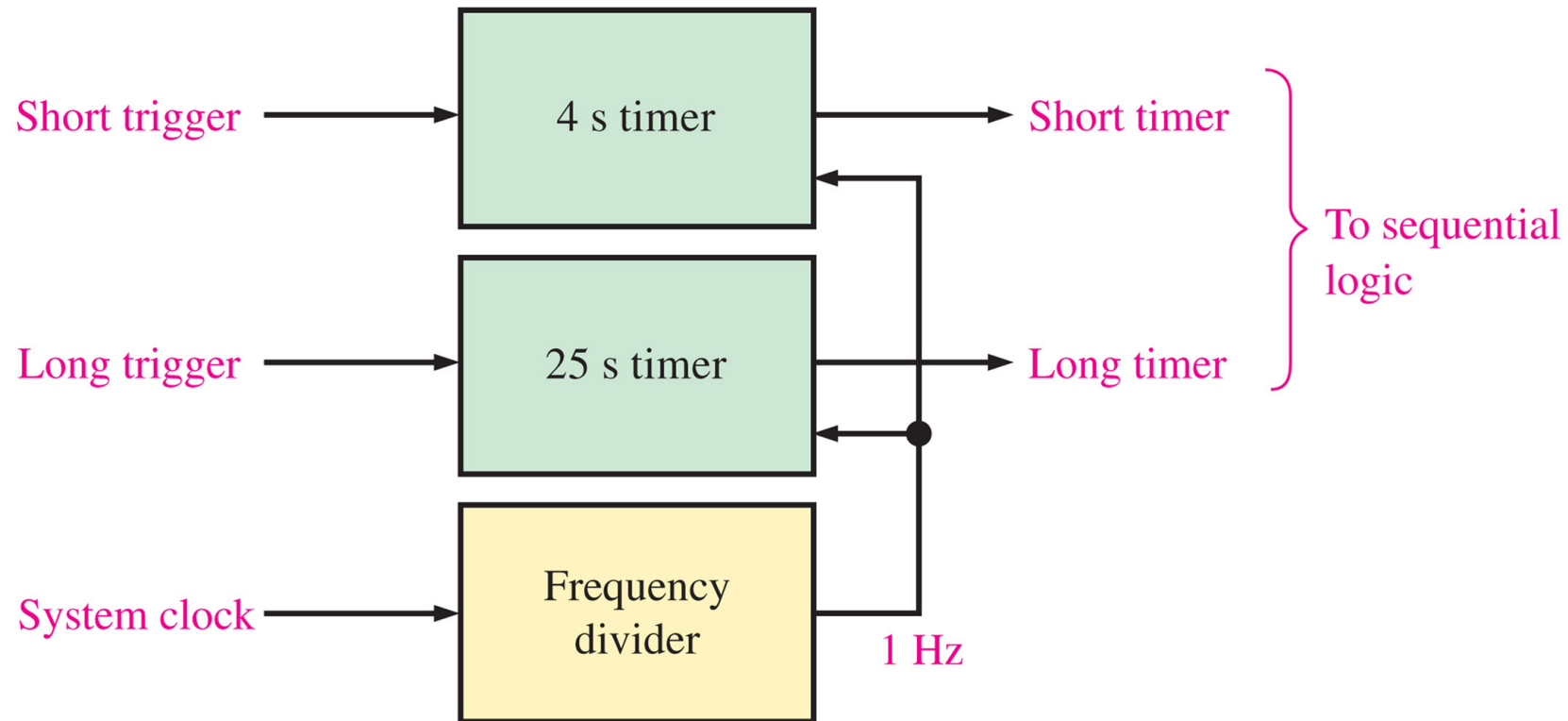
- Last lecture, we designed the combinational logic block
- This lecture, we will design the sequential logic block



# Combinational Logic block



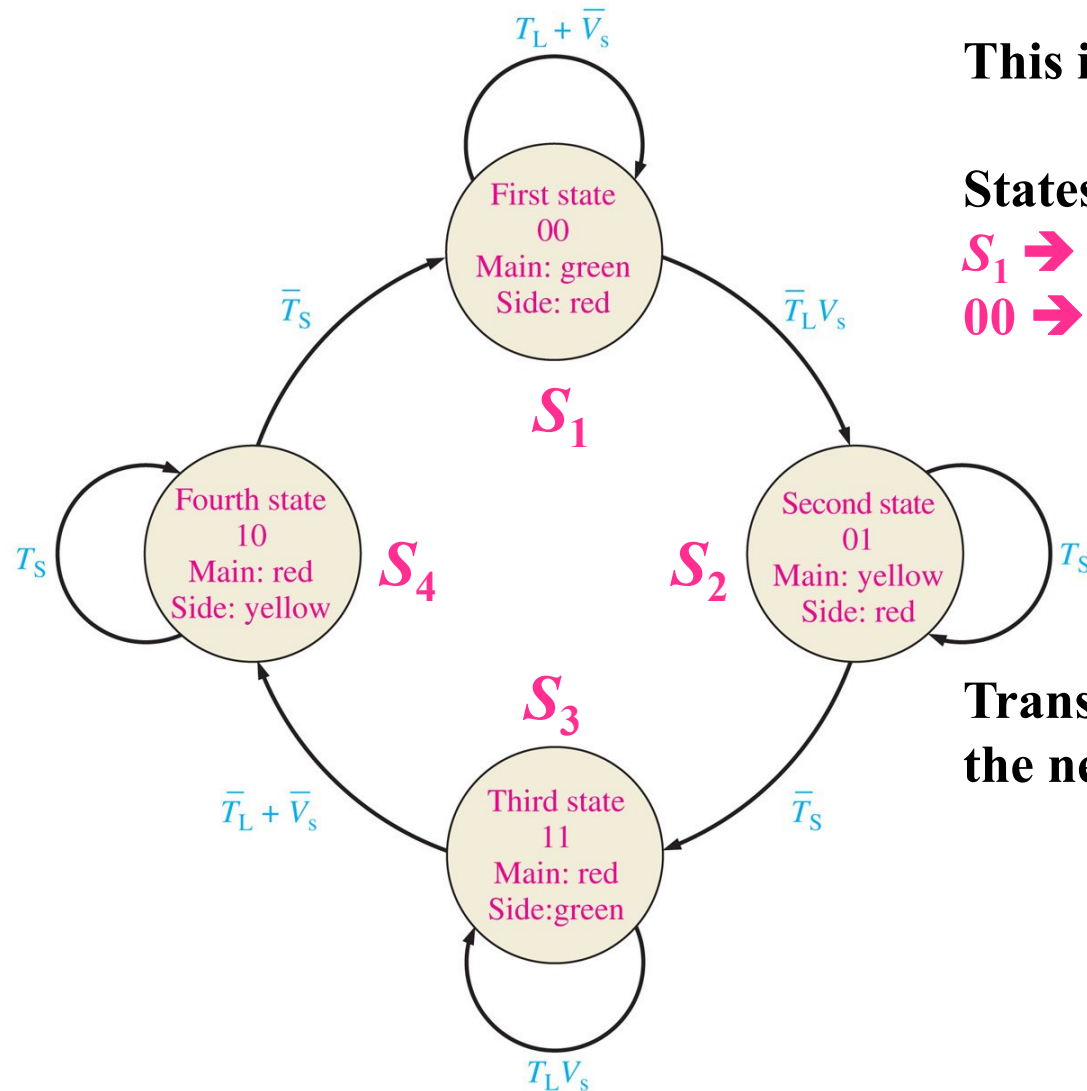
We skip this for now



This is actually a 2-bit Gray Code counter

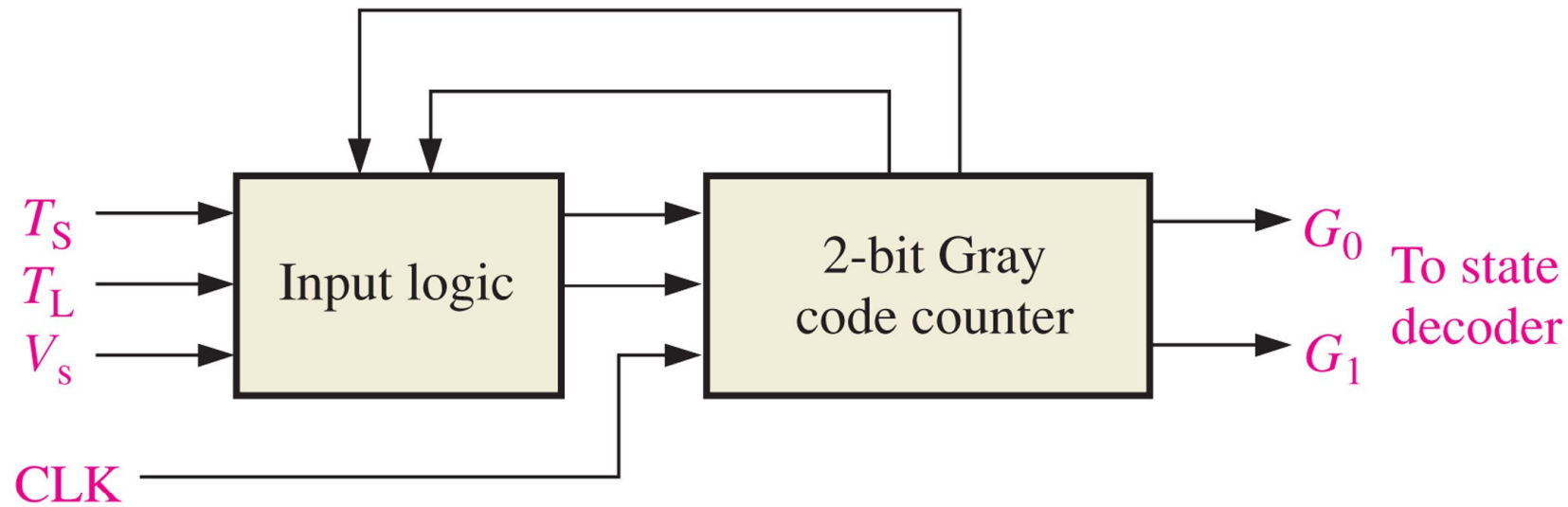
States:

$S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$   
 $00 \rightarrow 01 \rightarrow 11 \rightarrow 10$



Transition (counting) from one state (number) to the next is determined by  $T_L$ ,  $T_s$ , and  $V_s$

Outputs from the input logic provide the inputs to the counter so it follows correct count sequence

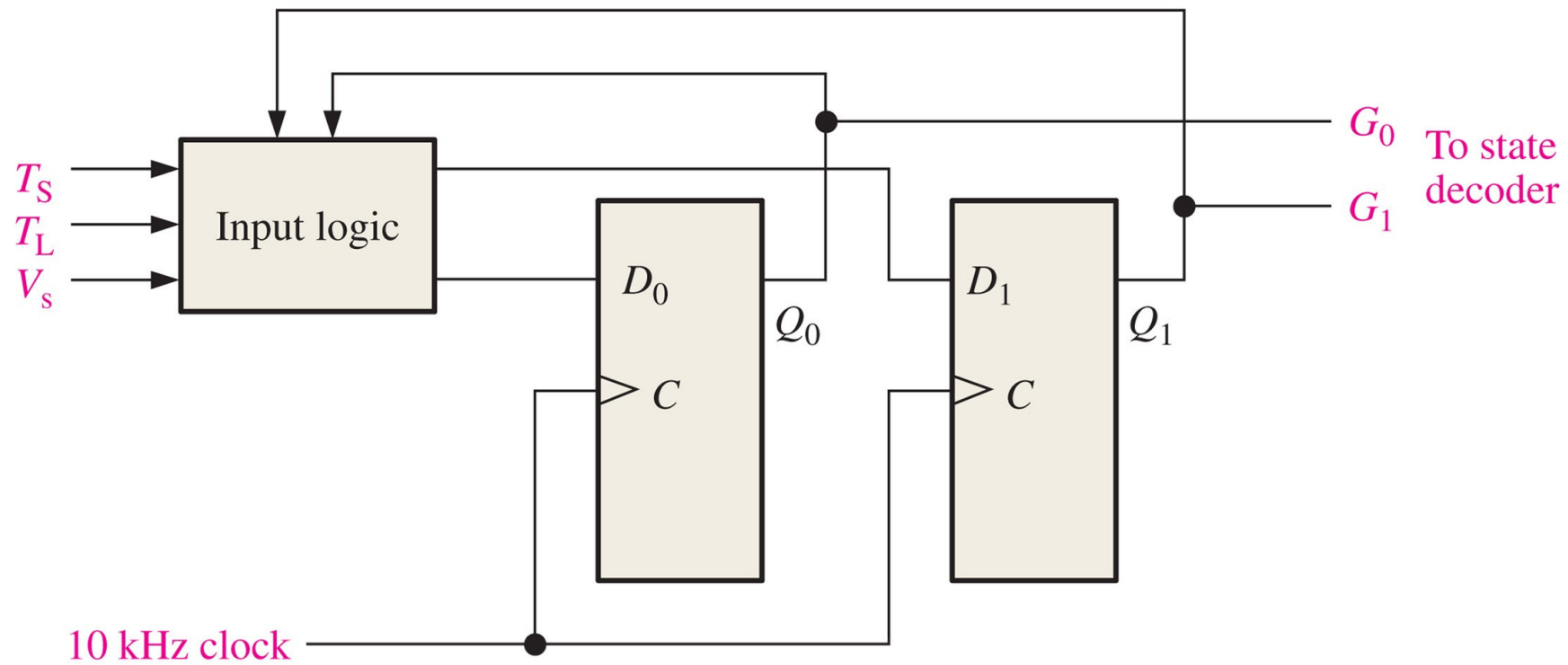


$T_S$  : Short timer (4 s)

$T_L$  : Long timer (25 s)

$V_s$  : Vehicle sensor for the side street

Sequential logic diagram with two  $D$  flip-flops used to implement the 2-bit Gray code counter

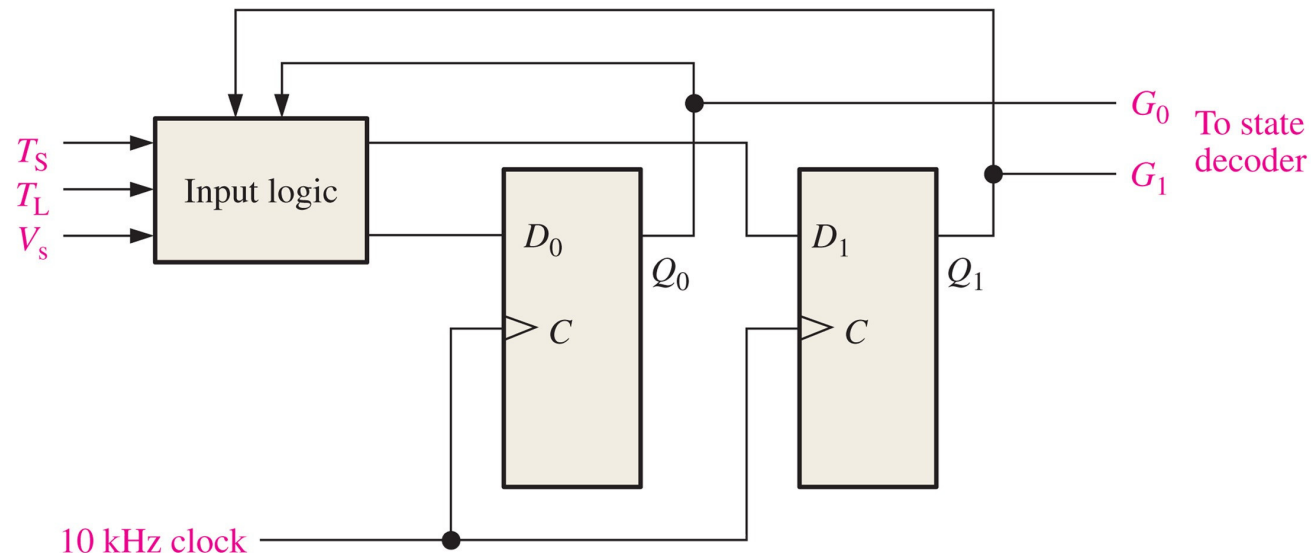


**The concept of "State" is very important in sequential logic**

**Correct sequential design requires correct "State diagram"**

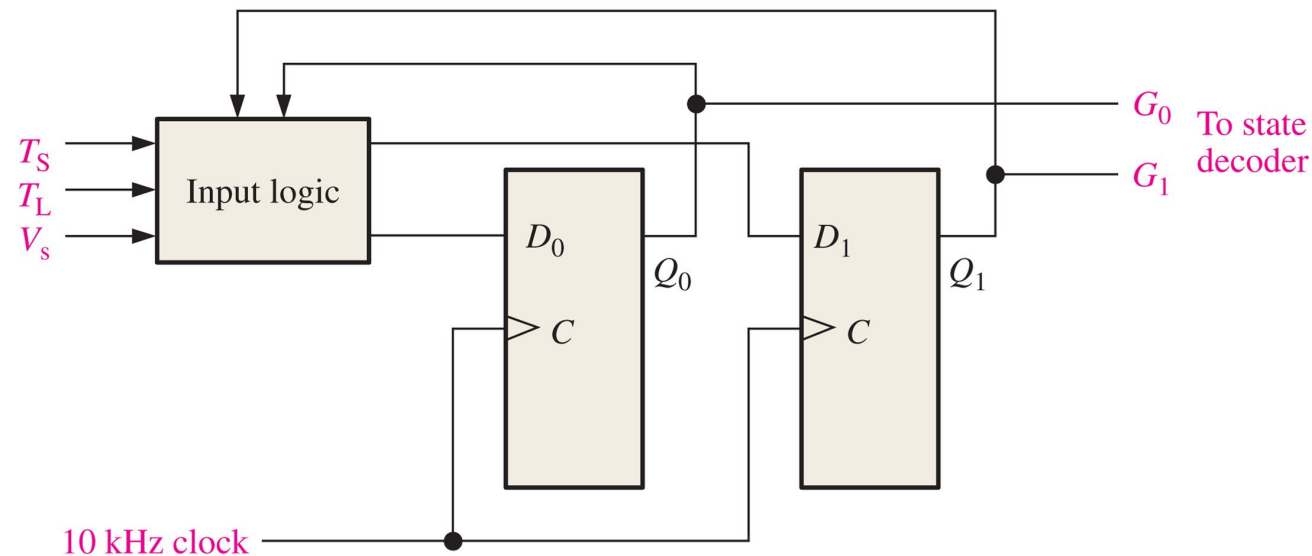
**Each state has a unique identifier code**

**Each state code refers to the combined output values of all the flip flops in the state register (e.g.,  $Q_1Q_0 = 01$ , means state 01).**



**Present State** refers to the output values of the flip flops within the present (current) clock cycle

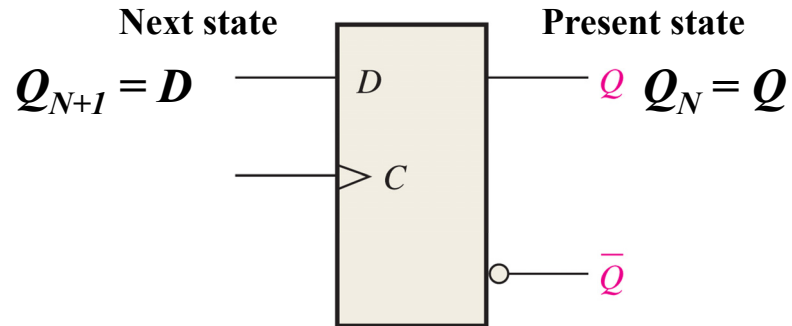
**Next State** refers to the output values of the flip flops that will appear in the coming (next) clock cycle



# D flip-flop state transition

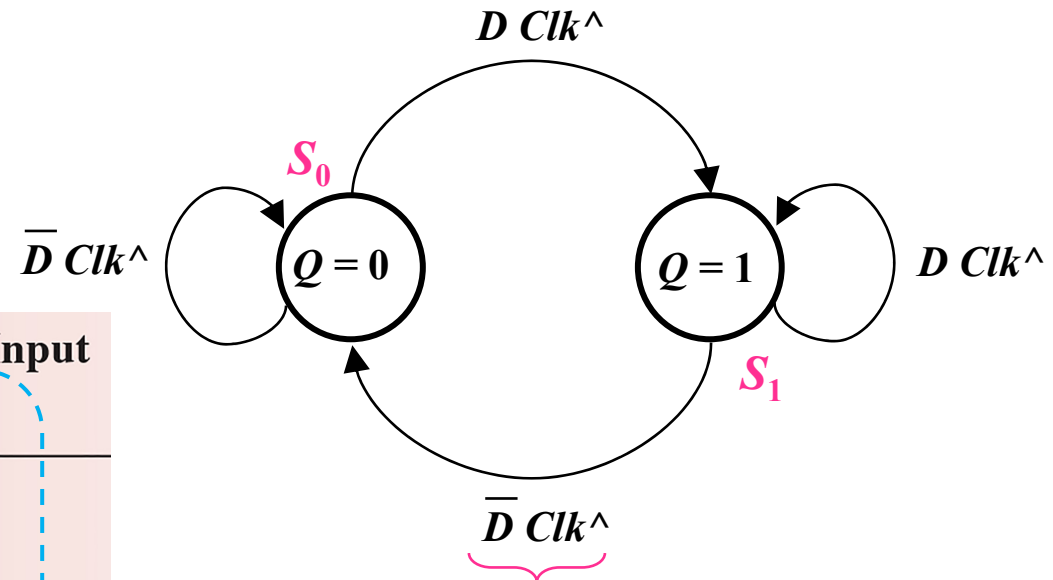
**Present state:**  $Q_N$  the output within the present clock cycle

**Next state:**  $Q_{N+1}$  the output within the next clock cycle.



**State diagram:**

$Clk^\wedge$  is TRUE (binary 1) if  
clock edge has arrived.

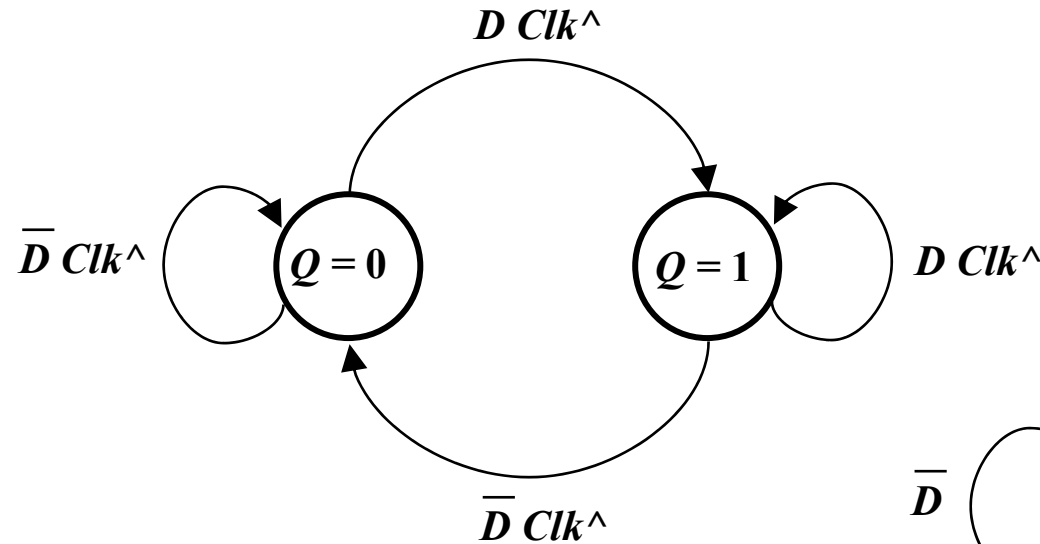


**State transition table**

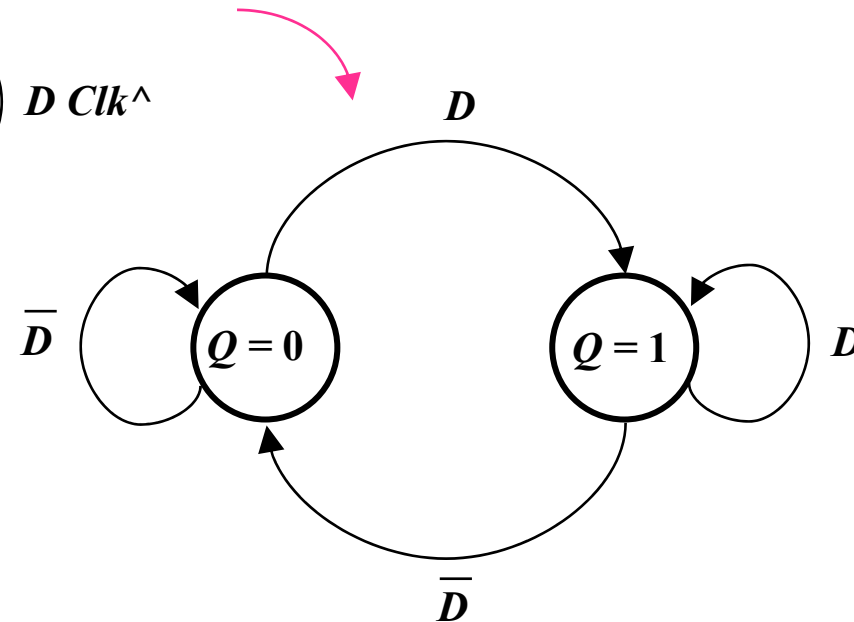
| Output Transitions |   | Flip-Flop Input |     |
|--------------------|---|-----------------|-----|
| $Q_N$              |   | $Q_{N+1}$       | $D$ |
| 0                  | → | 0               | 0   |
| 0                  | → | 1               | 1   |
| 1                  | → | 0               | 0   |
| 1                  | → | 1               | 1   |

*Transition from state  $S_1$  to state  $S_0$  occurs  
when this expression becomes TRUE  
(meaning  $D = 0$  AND  $Clk^\wedge = 1$ )*

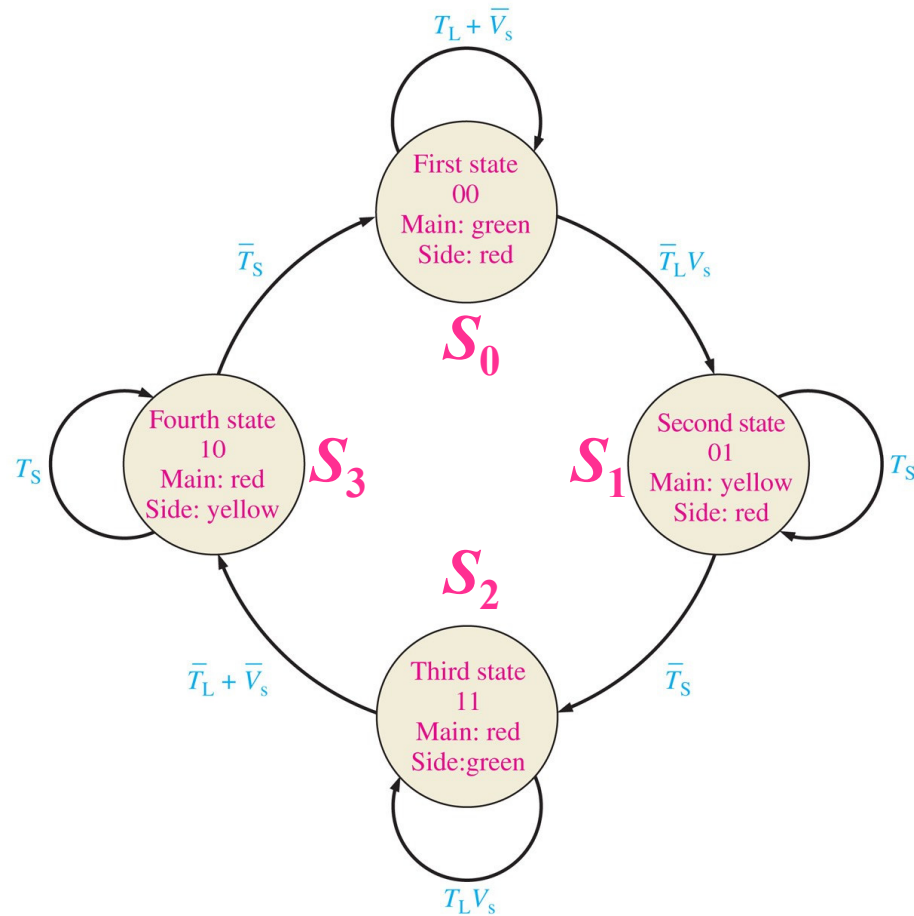
# D flip-flop state transition



*Typically  $Clk^\wedge$  is not written explicitly but we understand that it is there in the expression (implicit; hidden)*



# 2-bit Gray Code Counter



**TABLE 7-6**

Next-state table for the counter.

| Present State |       | Next State |       | Input Conditions        |
|---------------|-------|------------|-------|-------------------------|
| $Q_1$         | $Q_0$ | $Q_1$      | $Q_0$ |                         |
| 0             | 0     | 0          | 0     | $T_L + \bar{V}_s$       |
| 0             | 0     | 0          | 1     | $\bar{T}_L V_s$         |
| 0             | 1     | 0          | 1     | $T_s$                   |
| 0             | 1     | 1          | 1     | $\bar{T}_s$             |
| 1             | 1     | 1          | 1     | $T_L V_s$               |
| 1             | 1     | 1          | 0     | $\bar{T}_L + \bar{V}_s$ |
| 1             | 0     | 1          | 0     | $T_s$                   |
| 1             | 0     | 0          | 0     | $\bar{T}_s$             |

# 2-bit Gray Code Counter



| Output Transitions |           | Flip-Flop Input |
|--------------------|-----------|-----------------|
| $Q_N$              | $Q_{N+1}$ | $D$             |
| 0                  | → 0       | 0               |
| 0                  | → 1       | 1               |
| 1                  | → 0       | 0               |
| 1                  | → 1       | 1               |



Fill out FF inputs column ( $D_1$   $D_0$ ) using state transition table of  $D$  flip flop.

Or simply copy the next state columns

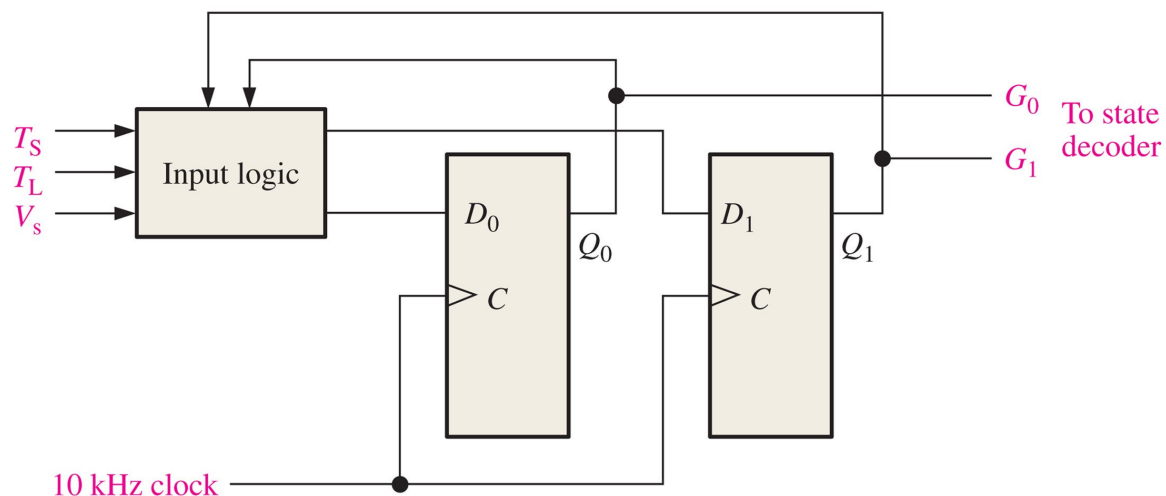


**TABLE 7-6**

Next-state table for the counter.

| Present State |       | Next State |       | Input Conditions        | FF Inputs |       |
|---------------|-------|------------|-------|-------------------------|-----------|-------|
| $Q_1$         | $Q_0$ | $Q_1$      | $Q_0$ |                         | $D_1$     | $D_0$ |
| 0             | 0     | 0          | 0     | $T_L + \bar{V}_s$       | 0         | 0     |
| 0             | 0     | 0          | 1     | $\bar{T}_L V_s$         | 0         | 1     |
| 0             | 1     | 0          | 1     | $T_S$                   | 0         | 1     |
| 0             | 1     | 1          | 1     | $\bar{T}_S$             | 1         | 1     |
| 1             | 1     | 1          | 1     | $T_L V_s$               | 1         | 1     |
| 1             | 1     | 1          | 0     | $\bar{T}_L + \bar{V}_s$ | 1         | 0     |
| 1             | 0     | 1          | 0     | $T_S$                   | 1         | 0     |
| 1             | 0     | 0          | 0     | $\bar{T}_S$             | 0         | 0     |

# 2-bit Gray Code Counter



**Treat present state as the truth table inputs  
And next state and flip-flop inputs as the  
truth table outputs**

**Write  $D_0$  and  $D_1$  in terms of present state  
and the input conditions using SOP or POS**

$$D_0 = \bar{Q}_1 \bar{Q}_0 \bar{T}_L V_S + \bar{Q}_1 Q_0 + Q_1 Q_0 T_L V_S$$

$$= \bar{G}_1 \bar{G}_0 \bar{T}_L V_S + \bar{G}_1 G_0 + G_1 G_0 T_L V_S$$

$$D_1 = \bar{Q}_1 Q_0 \bar{T}_S + Q_1 Q_0 T_L V_S + Q_1 Q_0 \bar{T}_L$$

$$+ Q_1 Q_0 \bar{V}_S + Q_1 \bar{Q}_0 T_S$$

$$= \bar{G}_1 G_0 \bar{T}_S + G_1 G_0 T_L V_S + G_1 G_0 \bar{T}_L$$

$$+ G_1 G_0 \bar{V}_S + G_1 \bar{G}_0 T_S$$

**TABLE 7-6**

Next-state table for the counter.

| Present State |       | Next State |       | Input Conditions        | FF Inputs |       |
|---------------|-------|------------|-------|-------------------------|-----------|-------|
| $Q_1$         | $Q_0$ | $Q_1$      | $Q_0$ |                         | $D_1$     | $D_0$ |
| 0             | 0     | 0          | 0     | $T_L + \bar{V}_S$       | 0         | 0     |
| 0             | 0     | 0          | 1     | $\bar{T}_L V_S$         | 0         | 1     |
| 0             | 1     | 0          | 1     | $T_S$                   | 0         | 1     |
| 0             | 1     | 1          | 1     | $\bar{T}_S$             | 1         | 1     |
| 1             | 1     | 1          | 1     | $T_L V_S$               | 1         | 1     |
| 1             | 1     | 1          | 0     | $\bar{T}_L + \bar{V}_S$ | 1         | 0     |
| 1             | 0     | 1          | 0     | $T_S$                   | 1         | 0     |
| 1             | 0     | 0          | 0     | $\bar{T}_S$             | 0         | 0     |

# Sequential logic diagram

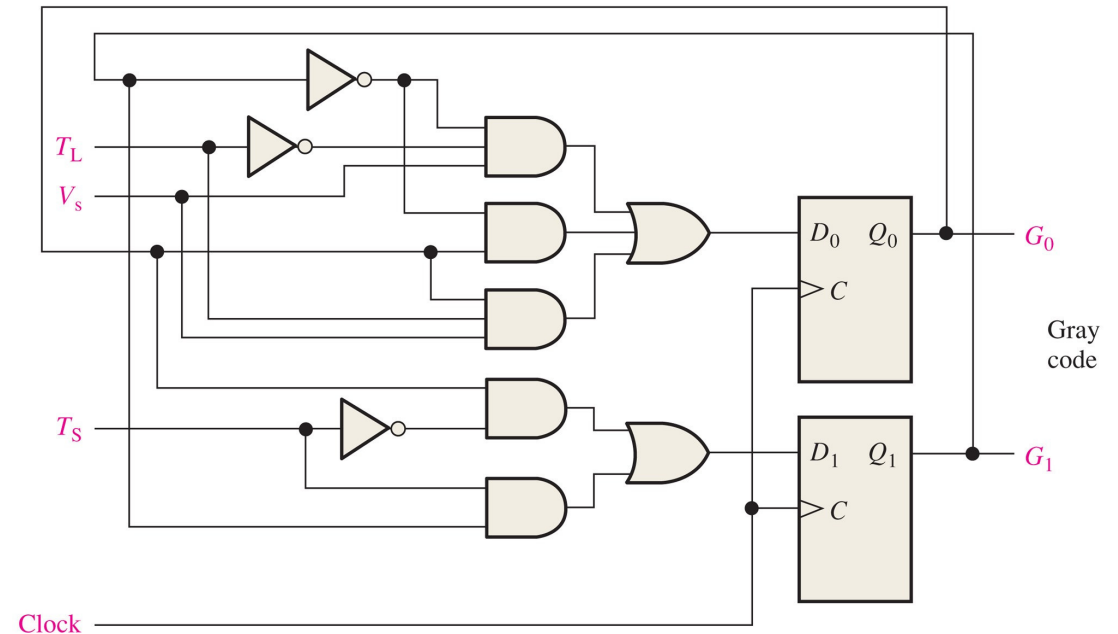
$$D_0 = \overline{G_1} \overline{G_0} \overline{T_L} V_S + \overline{G_1} G_0 + G_1 G_0 T_L V_S$$

$$D_1 = \overline{G_1} G_0 \overline{T_S} + G_1 G_0 T_L V_S + G_1 G_0 \overline{T_L}$$

$$+ G_1 G_0 \overline{V_S} + G_1 \overline{G_0} T_S$$

These expressions can be further minimized using Boolean Algebra

We will study minimization in future classes





香港科技大學  
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

*Thank you for listening !*

